

1 项目概述与架构

Spark Push 03_msg_subset 实时消息推送系统
零声教育 C++Linux课程 - 消息子集模块

1.1. 项目概述

1.1.1 03相比02的核心增量

02_auth_subset (认证子集) 实现了基础的 WebSocket 连接和身份认证，而 03_msg_subset (消息子集) 在此基础上新增了完整的消息推送能力：

新增服务

服务	说明	端口
Job	Kafka消费者，负责从消息队列拉取任务并推送到Comet	-

新增模块

模块	位置	说明
Kafka Producer	common/kafka_producer.cpp	Logic发送消息到Kafka队列
Kafka Consumer	common/kafka_consumer.cpp	Job从Kafka队列消费消息
Comet gRPC服务端	comet/comet_grpc_service.cpp	接收Job的下行推送请求
消息助手	logic/message_helper.cpp	构造和处理消息相关逻辑

新增功能

- 1. 一对一单聊
 - WebSocket/HTTP双通道发送消息
 - 消息ACK确认机制
 - 客户端消息ID (client_msg_id) 支持
- 2. Kafka消息队列
 - Logic → Kafka：异步投递推送任务
 - Kafka → Job：Pull模式消费
 - 支持消息持久化、重试、幂等
- 3. Redis路由增强
 - 支持一用户多连接 (HASH结构)

- 支持跨Comet路由（提取comet_id并去重）
- TTL自动过期机制

4. 保活机制

- 客户端：定时发送Ping帧
 - Comet：收到Ping/Pong/业务消息时刷新Redis TTL
 - Comet：分片扫描空闲连接并踢下线
-

1.1.2 核心功能

✓ 消息发送

- WebSocket上行消息（客户端 → Comet → Logic）
- HTTP接口发送（客户端 → Logic）

✓ 消息推送

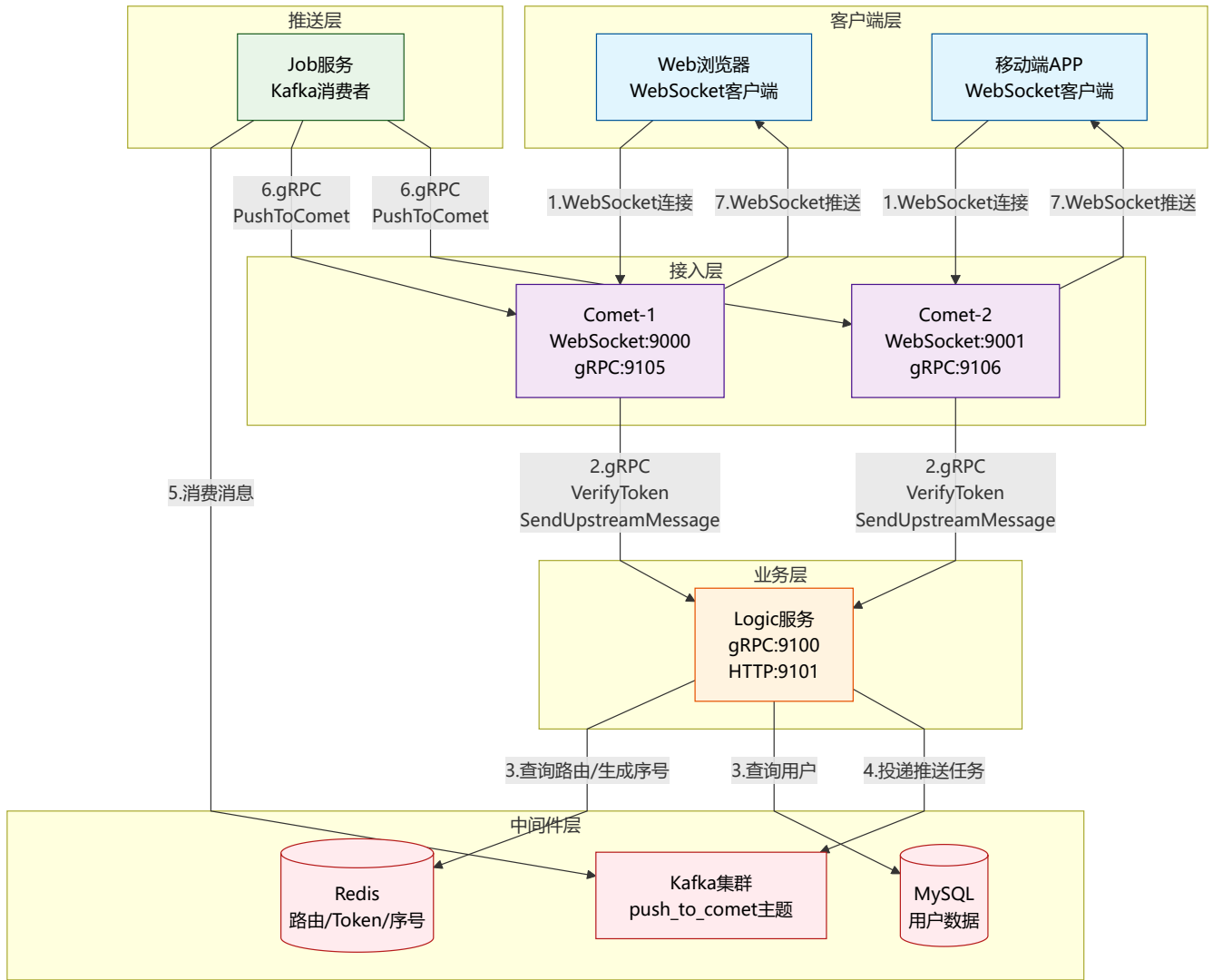
- Kafka异步队列（Logic → Kafka → Job → Comet）
- 支持多设备/多连接同时推送（一用户多连接）
- 跨Comet节点路由

✓ 连接管理

- WebSocket握手鉴权（Token验证）
 - 在线路由注册（Redis HASH结构）
 - 保活机制（Ping/Pong + TTL刷新）
 - 空闲连接自动清理
-

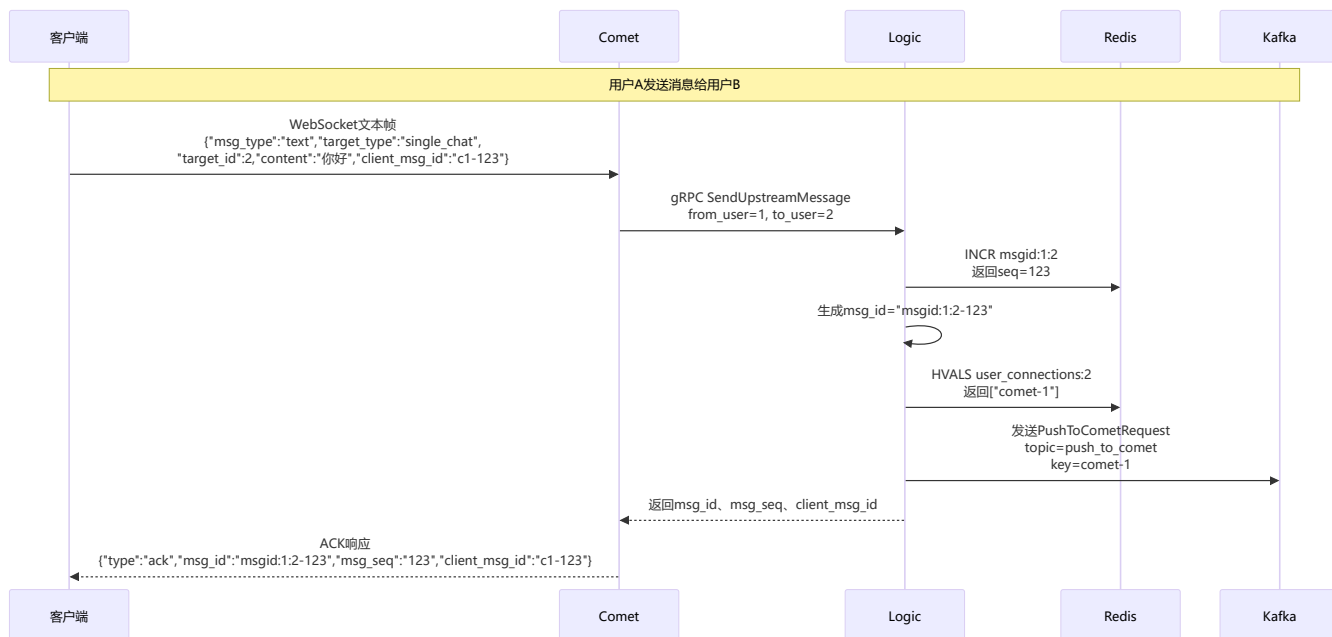
1.2. 系统架构

1.2.1 服务拓扑

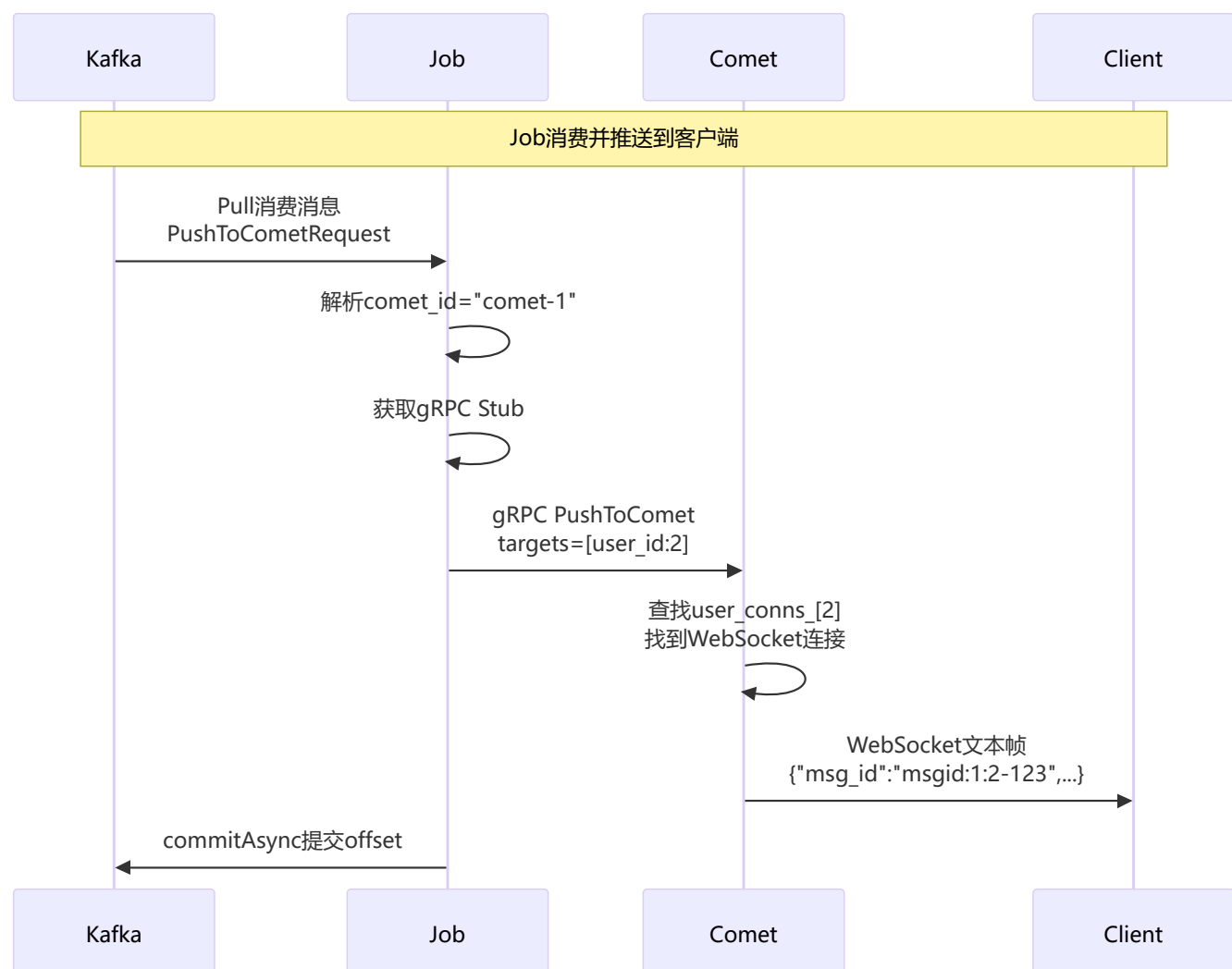


1.2.2 数据流向

上行消息（客户端发送）



下行推送（服务端推送）



1.2.3 服务职责

Comet（长连接网关）

职责	说明
WebSocket服务端	接受客户端连接，处理握手、帧解析、心跳
gRPC客户端	调用Logic进行Token验证、上行消息转发
gRPC服务端	接收Job的下行推送请求
连接管理	维护 <code>user_conns_</code> 映射，支持按用户推送
保活机制	处理Ping/Pong帧，刷新Redis TTL，扫描空闲连接

Logic（业务逻辑层）

职责	说明
gRPC服务端	提供VerifyToken、SendUpstreamMessage、UserOffline接口
HTTP服务端	提供注册、登录、发送消息、查询会话等REST API
消息路由	查询Redis路由，决定消息投递到哪些Comet节点
序号生成	通过Redis INCR生成全局唯一递增的msg_id
Kafka生产者	将推送任务异步投递到Kafka队列

Job（推送调度）

职责	说明
Kafka消费者	从 <code>push_to_comet</code> 主题拉取推送任务
Stub管理	动态管理多个Comet节点的gRPC连接
推送分发	根据comet_id调用对应Comet的gRPC接口
异步处理	使用线程池处理RPC调用，避免阻塞消费线程

1.3. 编译与启动

1.3.1 编译构建

```
cd spark_push/03_msg_subset

# 创建构建目录
mkdir -p build && cd build
```

```
# 生成Makefile
cmake ..

# 编译（使用所有CPU核心）
make -j$(nproc)

# 编译产物
ls -lh comet/comet_server logic/logic_server job/job_server
```

1.3.2 启动服务

步骤1：启动Redis

```
# 启动Redis（默认端口6379）
redis-server --port 6379 --daemonize yes

# 验证
redis-cli ping

# 输出：PONG
```

步骤2：启动Kafka

如果没有启动kafka，请参考第一节文档。

```
# 确保kafka已经启动
sudo lsof -i:9092

# 后续如果出现 链路不通时，可以用命令方式测试kafka是否正常
# 启动push_to_comet主题消费者
bin/kafka-console-consumer.sh \
  --topic push_to_comet \
  --from-beginning \
  --bootstrap-server localhost:9092
```

⚠ 注意：如果Kafka启动报错，需重新初始化。

步骤3：启动Comet

```
cd spark_push/03_msg_subset/build

# 启动Comet服务（WebSocket网关）
./comet/comet_server ../conf/comet.conf

# 日志输出示例
# [INFO] Comet server starting...
# [INFO] Listen on 0.0.0.0:9000
# [INFO] gRPC server listen on 0.0.0.0:9105
```

步骤4: 启动Logic

```
# 新终端
cd spark_push/03_msg_subset/build

./logic/logic_server ../conf/logic.conf

# 日志输出示例
# [INFO] Logic gRPC server started on 0.0.0.0:9100
# [INFO] HTTP server started on 0.0.0.0:9101
```

步骤5: 启动Job

```
# 新终端
cd spark_push/03_msg_subset/build

./job/job_server ../conf/job.conf

# 日志输出示例
# [INFO] Kafka consumer initialized
# [INFO] Job service started, waiting for messages...
```

步骤6: 重置MySQL

可以复制: 03_msg_subset/sql/schema.sql里的sql语句在mysql命令行执行:

```
-- 自动创建数据库并初始化用户表
-- 支持重复执行（幂等）
DROP DATABASE IF EXISTS spark_push;
CREATE DATABASE IF NOT EXISTS `spark_push`
  DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE `spark_push`;

-- 用户表: 账户与密码（演示版使用 MD5，生产环境应使用 bcrypt + 盐值）
CREATE TABLE IF NOT EXISTS `user` (
  `id` BIGINT NOT NULL AUTO_INCREMENT,
  `account` VARCHAR(64) NOT NULL UNIQUE,
  `name` VARCHAR(64) NOT NULL DEFAULT '',
  `password_hash` VARCHAR(128) NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_account` (`account`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

步骤7: 重置redis

只允许测试环境这么做

```
# 我们默认是在db 0，清空当前redis，只需要测试环境如此操作
127.0.0.1:6379> FLUSHDB
OK
```

1.3.3 快速测试

测试1: 注册用户

```
# 注册用户1
curl -X POST http://localhost:9101/api/register \
  -H "Content-Type: application/json" \
  -d '{"account":"user1","password":"123456","name":"张三"}'

# 响应
{"code":0,"message":"ok","data":{"user_id":1,"token":"tk-1-1766055878","name":"张三"}}

# 注册用户2
curl -X POST http://localhost:9101/api/register \
  -H "Content-Type: application/json" \
  -d '{"account":"user2","password":"123456","name":"李四"}'

{"code":0,"message":"ok","data":{"user_id":2,"token":"tk-2-1766055898","name":"李四"}}
```

测试2: 登录获取Token

```
# 用户1登录
curl -X POST http://localhost:9101/api/login \
  -H "Content-Type: application/json" \
  -d '{"account":"user1","password":"123456"}'

# 响应（记录token）
{"code":0,"message":"ok","data":{"user_id":1,"token":"tk-1-1766055930","name":"张三"}}

# 用户2登录
curl -X POST http://localhost:9101/api/login \
  -H "Content-Type: application/json" \
  -d '{"account":"user2","password":"123456"}'

# 响应
{"code":0,"message":"ok","data":{"user_id":2,"token":"tk-2-1766055973","name":"李四"}}
```

测试3: WebSocket连接 (带保活)

⚠ **重要:** 必须使用 `--ping-interval` 参数保持连接活跃!

```
# 安装websocat (如果未安装)
# cargo install websocat
# 或 wget https://github.com/vi/websocat/releases/download/v1.11.0/websocat_linux64

# 用户1连接 (带30秒保活间隔)
# --ping-interval 30 每30秒发送一次保活请求, 然后需要服务器在40秒内应答
websocat "ws://localhost:9000/ws?token=tk-1-1766060955" \
  --ping-interval 30 \
  --ping-timeout 40 -E

# 用户2连接 (新终端)
websocat "ws://localhost:9000/ws?token=tk-2-1766060994" \
  --ping-interval 30 \
  --ping-timeout 40 -E
```

测试4: websocket发送消息

在用户1的websocat终端输入:

```
{"msg_type": "text", "target_type": "single_chat", "target_id": 2, "content": "你好, 李四! ", "client_msg_id": "c1-001"}
```

预期结果:

1. 用户1收到ACK:

```
{"type": "ack", "msg_id": "msgid:1:2-1", "msg_seq": 1, "client_msg_id": "c1-001"}
```

2. 用户2收到消息:

```
{
  "client_msg_id": "c1-001",
  "content": "你好, 李四! ",
  "create_time": 1766056082456,
  "from_user_id": 1,
  "msg_id": "msgid:1:2-1",
  "msg_seq": 1,
  "msg_type": "text",
  "target_id": 2,
  "target_type": "single_chat"
}
```

测试5: HTTP 发送消息

```
curl -X POST http://localhost:9101/api/message/send \
-H "Authorization: Bearer tk-1-1766060955" \
-H "Content-Type: application/json" \
-d '{
  "msg_type": "text",
  "target_type": "single_chat",
  "target_id": 2,
  "content": "HTTP消息（来自 HTTP）",
  "client_msg_id": "c1-456"
}'
```

响应

```
{"code":0,"message":"ok","data":{"msg_id":"msgid:1:2-2","msg_seq":2,"session_id":"s_1:2"}}
```

2 协议与鉴权机制

消息协议设计、WebSocket连接鉴权、保活机制详解

2.1. 消息协议设计

2.1.1 msg_id设计原理

为什么需要服务端生成ID?

在分布式系统中，消息需要**全局唯一**且**严格递增**的标识，用于：

- ✓ 消息排序（按序号展示聊天记录）
- ✓ 消息去重（防止重复消息）
- ✓ 未读计数（基于序号差计算未读数）
- ✓ 增量同步（客户端拉取"seq > 100"的新消息）

客户端生成的ID无法保证全局顺序性，因此必须由服务端统一分配。

msg_id格式

msgid:{user1}:{user2}-{seq}

示例: msgid:1:2-123

组成部分：

- `msgid`: 固定前缀
- `user1`: 较小的`user_id` (保证A→B和B→A使用同一计数器)
- `user2`: 较大的`user_id`
- `seq`: Redis INCR原子递增的序号

生成流程

```
// logic/grpc_service.cpp (SendUpstreamMessage方法中)

// 1. 确保user1 < user2 (保证双向会话使用同一序号计数器)
int64_t from_user = request->from_user_id(); // 发送方
int64_t to_user = request->target_id();      // 接收方
int64_t user1 = std::min(from_user, to_user);
int64_t user2 = std::max(from_user, to_user);

// 2. Redis INCR获取严格递增序号
int64_t seq = 0;
if (!redis_store->NextSingleMsgId(user1, user2, &seq)) {
    return ERROR("generate msg_id failed");
}
// NextSingleMsgId内部执行: INCR msgid:1:2 → 返回123

// 3. 拼接完整msg_id
std::string msg_id = "msgid:" + std::to_string(user1) + ":" +
                    std::to_string(user2) + "-" + std::to_string(seq);
// 结果: msgid:1:2-123
```

Redis命令演示

```
# 用户1给用户2发第一条消息
127.0.0.1:6379> INCR msgid:1:2
(integer) 1

# 用户1给用户2发第二条消息
127.0.0.1:6379> INCR msgid:1:2
(integer) 2

# 用户2给用户1发消息 (同一个key! 保证会话全局递增)
127.0.0.1:6379> INCR msgid:1:2
(integer) 3

# 并发安全: INCR是原子操作
127.0.0.1:6379> INCR msgid:1:2
(integer) 4
127.0.0.1:6379> INCR msgid:1:2
(integer) 5

# 无重复, 严格递增 ✓
```

为什么不用UUID?

方案	优点	缺点	是否采用
Redis INCR	- 严格递增 - 可读性强 - 支持未读计数	依赖Redis	✅ 本项目采用
UUID	- 全局唯一 - 无需中心化	- 无序性 - 存储开销大 - 无法未读计数	❌
雪花算法	- 趋势递增 - 高性能	- 需时钟同步 - 不适合未读计算	❌

2.1.2 client_msg_id的作用

格式设计

```
c-{timestamp}-{random}
```

示例：c-1734567890-abc123

客户端生成，用于：

- ✅ **ACK匹配**：客户端发送消息后立即显示"发送中"，收到ACK后更新为"已发送"
- ✅ **去重**：防止客户端重复发送
- ✅ **状态管理**：客户端维护pending消息列表

使用场景

场景1：发送消息时立即显示

```
// 客户端代码示例
function sendMessage(content) {
  // 1. 生成client_msg_id
  const clientMsgId = 'c-' + Date.now() + '-' + Math.random().toString(36).substr(2);

  // 2. 立即在UI中显示"发送中"
  displayMessage({
    client_msg_id: clientMsgId,
    content: content,
    status: 'sending' // 发送中状态
  });

  // 3. 发送到服务端
  ws.send(JSON.stringify({
    msg_type: 'text',
    target_type: 'single_chat',
    target_id: 2,
```

```

        content: content,
        client_msg_id: clientMsgId
    }));
}

// 收到ACK时更新状态
ws.onmessage = function(event) {
    const data = JSON.parse(event.data);

    if (data.type === 'ack') {
        // 根据client_msg_id找到对应消息，更新状态
        updateMessageStatus(data.client_msg_id, 'sent', data.msg_id);
        // UI更新: 发送中 ⌚ → 已发送 ✓
    }
};

```

场景2: 防止重复发送

```

const pendingMessages = new Map();

function sendMessage(content) {
    const clientMsgId = 'c-' + Date.now() + '-' + Math.random().toString(36).substr(2);

    // 检查是否已发送
    if (pendingMessages.has(clientMsgId)) {
        console.warn('Message already sent:', clientMsgId);
        return;
    }

    pendingMessages.set(clientMsgId, {content, timestamp: Date.now()});
    ws.send(JSON.stringify({...}));

    // 60秒后自动清理（避免内存泄漏）
    setTimeout(() => {
        pendingMessages.delete(clientMsgId);
    }, 60000);
}

```

2.1.3 msg_seq序号提取

设计原则

⚠ 重要: msg_seq 不是客户端生成的，而是从服务端的 msg_id 中提取的序号。

```
// 错误做法（客户端生成）❌
msg_seq = 1, 2, 3... // 客户端自己维护序号
// 问题：无法保证全局顺序，不同客户端可能重复

// 正确做法（从msg_id提取）✅
msg_id = "msgid:1:2-123"
msg_seq = 123 // 从msg_id提取的序号
// 优点：全局唯一、严格递增、可用于排序和未读计数
```

提取方法

```
// logic/grpc_service.cpp（补充消息内容时）
std::string msg_id = "msgid:1:2-123";
int64_t seq = 123; // 已从Redis INCR获取

// 将seq填充到消息中
try {
    auto j = nlohmann::json::parse(final_json);
    j["msg_id"] = msg_id;
    j["msg_seq"] = seq; // 关键：直接使用INCR返回的seq
    j["create_time"] = now_ms;
    final_json = j.dump();
} catch (...) {}
```

用途

1. 消息排序

```
-- MySQL查询（如果消息落库）
SELECT * FROM message
WHERE session_id='s_1:2'
ORDER BY msg_seq -- 按序号排序
LIMIT 20;
```

2. 未读计数

```
// 本项目实现：Redis递增计数器
int64_t unread = 0;
redis_store->GetUnreadCount(user_id, session_id, &unread);

// 扩展实现（基于已读位置）：
// unread = session_last_seq - user_read_seq
// 例如：123 - 118 = 5条未读
```

3. 增量拉取

```
-- 客户端拉取"seq > 100"之后的新消息
SELECT * FROM message
WHERE session_id='s_1:2' AND msg_seq > 100
ORDER BY msg_seq
LIMIT 20;
```

2.1.4 完整对比表

维度	msg_id	client_msg_id	msg_seq
生成方	服务端 (Logic)	客户端	服务端 (从msg_id提取)
格式	msgid:1:2-123	c-1734567890-abc	整数 (123)
唯一性	全局唯一	客户端内唯一	会话内唯一且递增
是否必需	✅ 必需	⚠️ 推荐 (ACK匹配)	✅ 必需 (从msg_id提取)
用途1	全局标识、防重	ACK匹配	消息排序
用途2	日志追踪	客户端去重	未读计数
用途3	包含递增序号	状态管理	增量拉取

推荐使用模式

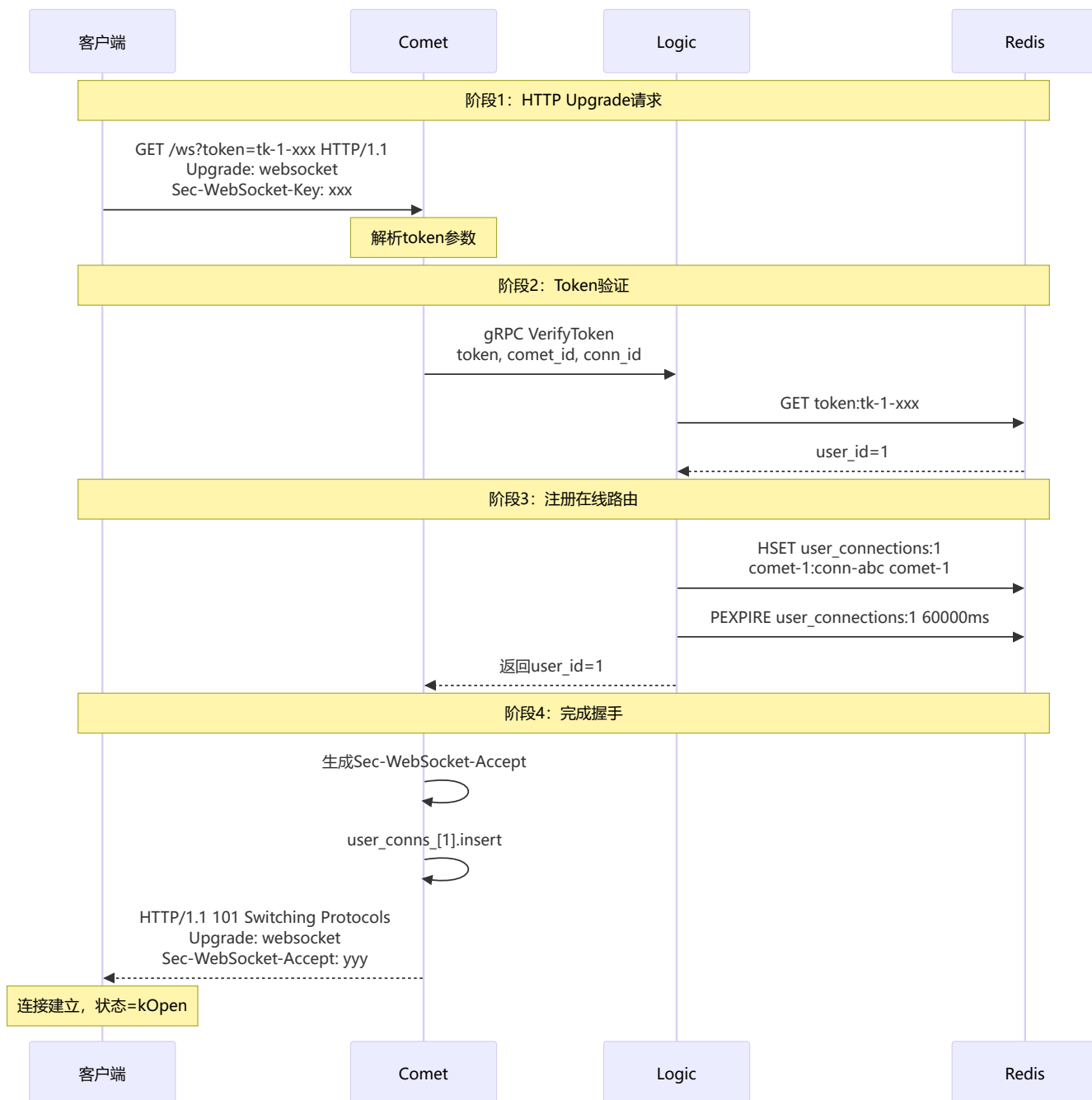
生产环境（完整版）：

```
{
  "msg_id": "msgid:1:2-123",           // 服务端生成（必需）
  "msg_seq": 123,                     // 从msg_id提取（必需）
  "client_msg_id": "c-1734567890"    // 客户端生成（推荐）
}
```

- ✅ 功能完整：排序、未读、ACK、去重
- ✅ 体验良好：发送消息立即显示

2.2. WebSocket连接与鉴权

2.2.1 握手流程



2.2.2 Token验证

Token生成（登录时）

```
// logic/http_server.cpp (HandleLogin方法中)

// 1. 验证用户名密码（略）

// 2. 生成token
std::string token = "tk-" + std::to_string(user.id) + "-" +
    std::to_string(CurrentTimeMillis() / 1000);
// 例如: tk-1-1734567890

// 3. 存入Redis（24小时有效期）
```

```
redis_store->SetToken(token, user.id, 86400);
// 执行: SETEX token:tk-1-1734567890 86400 1
```

Token验证 (握手时)

```
// logic/grpc_service.cpp (VerifyToken方法)

::grpc::Status LogicServiceImpl::VerifyToken(
    ::grpc::ServerContext*,
    const VerifyTokenRequest* request,
    VerifyTokenReply* response) {

    // 1. 从Redis查询token
    int64_t user_id = 0;
    if (!redis_store->GetUserIdByToken(request->token(), &user_id)) {
        return ERROR("invalid or expired token");
    }

    // 2. 注册在线路由
    redis_store->UpsertUserConnection(
        user_id,
        request->comet_id(), // 例如: comet-1
        request->conn_id(), // 例如: comet-1-1734567890-1
        60000 // TTL 60秒
    );

    // 3. 返回user_id
    response->set_user_id(user_id);
    SetError(response->mutable_error(), 0, "ok");
    return ::grpc::Status::OK;
}
```

2.2.3 连接上下文

ConnContext结构

```
// comet/comet_server.h

struct ConnContext {
    // 连接状态
    enum State {
        kHandshake, // 握手阶段: 等待WebSocket握手完成
        kOpen       // 已建立: WebSocket已升级, 可收发消息
    };

    State state{kHandshake}; // 当前状态 (默认握手中)
    int64_t user_id{0};      // 绑定的用户ID (鉴权成功后设置)
    std::string conn_id;     // 连接唯一标识
    int64_t last_active_ms{0}; // 最近活跃时间 (用于空闲检测)
};
```

状态转换



2.2.4 路由注册

Redis数据结构

```
Key:   user_connections:{user_id}
Type:  HASH
Field: {comet_id}:{conn_id}
Value: {comet_id}
TTL:   PEXPIRE 60000ms (可配置)
```

用户路由示例

```
# 用户1在comet-1有2个连接 (PC+手机)
127.0.0.1:6379> HGETALL user_connections:1
1) "comet-1:comet-1-1734567890-1"
2) "comet-1"
3) "comet-1:comet-1-1734567900-2"
4) "comet-1"

# 用户2跨Comet登录 (PC在comet-1, 手机在comet-2)
127.0.0.1:6379> HGETALL user_connections:2
1) "comet-1:comet-1-1734567910-3"
2) "comet-1"
3) "comet-2:comet-2-1734567920-1"
4) "comet-2"

# 查询TTL
127.0.0.1:6379> PTTL user_connections:1
(integer) 58234 # 剩余58.234秒
```

UpsertUserConnection实现

```
// logic/redis_store.cpp

bool RedisStore::UpsertUserConnection(int64_t user_id,
                                     const std::string& comet_id,
                                     const std::string& conn_id,
                                     int64_t ttl_ms) {

    auto guard = pool_->Acquire(0);
    redisContext* ctx = guard.get();
    if (!ctx) return false;

    std::string key = "user_connections:" + std::to_string(user_id);
    std::string field = comet_id + ":" + conn_id;

    // 1. HSET user_connections:1 comet-1:conn-abc comet-1
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "HSET %s %s %s", key.c_str(), field.c_str(), comet_id.c_str());

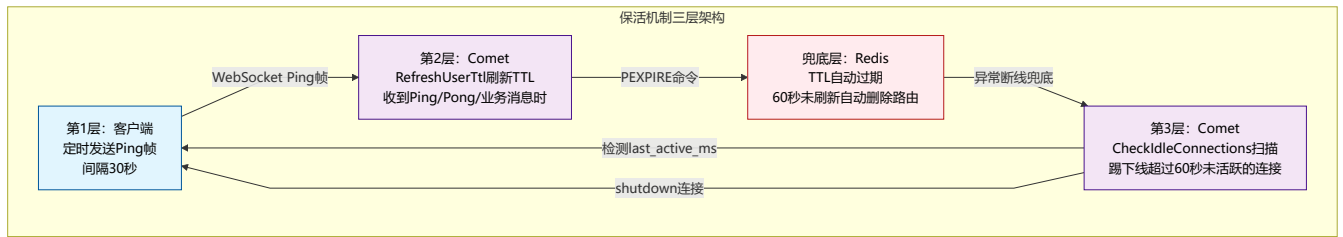
    if (!reply || reply->type == REDIS_REPLY_ERROR) {
        LOG_ERROR << "Redis HSET user_connections failed";
        if (reply) freeReplyObject(reply);
        return false;
    }
    freeReplyObject(reply);

    // 2. PEXPIRE user_connections:1 60000
    reply = (redisReply*)redisCommand(
        ctx, "PEXPIRE %s %lld", key.c_str(), static_cast<long long>(ttl_ms));

    if (reply) freeReplyObject(reply);
    return true;
}
```

2.3. 保活机制设计

2.3.1 三层保活协同



2.3.2 客户端保活实现

websocat测试 (推荐)

```
# ❌ 错误: 不带保活参数 (60秒后被踢下线)
websocat "ws://localhost:9000/ws?token=tk-1-xxx"

# ✅ 正确: 带保活参数 (每30秒发送Ping)
websocat "ws://localhost:9000/ws?token=tk-1-xxx" \
  --ping-interval 30 \
  --ping-timeout 10

# 参数说明:
# --ping-interval 30 : 每30秒发送一次Ping帧
# --ping-timeout 10 : Ping超时时间10秒 (未收到Pong则断开)
```

JavaScript客户端实现

```
class WebSocketClient {
  constructor(url) {
    this.ws = new WebSocket(url);
    this.pingInterval = null;

    this.ws.onopen = () => {
      console.log('Connected');
      // 启动心跳 (每30秒发送一次Ping)
      this.startPing();
    };

    this.ws.onmessage = (event) => {
      const data = JSON.parse(event.data);
      // 处理业务消息...
    };

    this.ws.onclose = () => {
      console.log('Disconnected');
      this.stopPing();
    };
  }

  startPing() {
    this.pingInterval = setInterval(() => {
      if (this.ws.readyState === WebSocket.OPEN) {
        // 发送Ping帧 (浏览器WebSocket API不支持直接发送Ping)
        // 解决方案: 发送业务层心跳消息
        this.ws.send(JSON.stringify({type: 'ping'}));
      }
    }, 3000); // 30秒
  }

  stopPing() {
    if (this.pingInterval) {

```

```

        clearInterval(this.pingInterval);
        this.pingInterval = null;
    }
}

// 使用
const client = new WebSocketClient('ws://localhost:9000/ws?token=tk-1-xxx');

```

⚠ **注意：**浏览器WebSocket API不支持直接发送RFC 6455标准的Ping帧，需要使用业务层心跳或依赖websocat等工具测试。

2.3.3 Comet TTL刷新

RefreshUserTtl实现

```

// comet/comet_server.cpp

void CometServer::RefreshUserTtl(int64_t user_id, const std::string& conn_id) {
    // 快速降级: Redis不可用时直接返回
    if (!redis_ready_ || user_id <= 0 || conn_id.empty()) {
        return;
    }

    // 1. 获取Redis连接
    auto guard = redis_pool_.Acquire(0);
    redisContext* ctx = guard.get();
    if (!ctx) return;

    // 2. 执行PEXPIRE命令（刷新TTL）
    std::string key = "user_connections:" + std::to_string(user_id);
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "PEXPIRE %s %lld",
        key.c_str(),
        static_cast<long long>(redis_ttl_ms_) // 默认60000ms
    );

    if (reply) freeReplyObject(reply);
}

```

调用时机

```

// comet/comet_server.cpp (HandleWebSocketFrame方法中)

void CometServer::HandleWebSocketFrame(...) {
    // ...解析帧头...

    uint8_t opcode = buf->peek()[0] & 0x0F;

    switch (opcode) {
        case 0x1: // 文本帧

```

```

        OnTextMessage(conn, ctx, payload);
        RefreshUserTtl(ctx.user_id, ctx.conn_id); // 刷新TTL ✓
        break;

    case 0x9: // Ping帧
        // 回复Pong
        conn->send(BuildWebSocketPongFrame(payload));
        RefreshUserTtl(ctx.user_id, ctx.conn_id); // 刷新TTL ✓
        break;

    case 0xA: // Pong帧
        // 记录日志
        LOG_INFO << "Recv pong from user " << ctx.user_id;
        RefreshUserTtl(ctx.user_id, ctx.conn_id); // 刷新TTL ✓
        break;
}

// 更新最近活跃时间
ctx.last_active_ms = CurrentTimeMillis();
conn->setContext(ctx);
}

```

2.3.4 空闲连接扫描

分片扫描机制

设计目标：避免全量扫描导致的CPU抖动和锁竞争。

核心思路：

1. 将所有用户按 `user_id % 10` 分到10个虚拟分片
2. 每次定时任务只扫描**一个分片**
3. 分片索引循环推进：0→1→2→...→9→0

复杂度：

- 单次扫描：O(N/10)
- 发现延迟上界：5秒 × 10 = 50秒

CheckIdleConnections实现

```

// comet/comet_server.cpp

void CometServer::CheckIdleConnections() {
    int64_t now_ms = CurrentTimeMillis();
    std::vector<TcpConnectionPtr> to_close;

    {
        std::lock_guard<std::mutex> lock(conns_mu_);

        // 只扫描"当前分片"的用户
    }
}

```

```

for (auto& kv : user_conns_) {
    int64_t user_id = kv.first;

    // 分片过滤：跳过不属于当前分片的用户
    if (idle_scan_shards_ > 1) {
        int shard = static_cast<int>(user_id % idle_scan_shards_);
        if (shard < 0) shard += idle_scan_shards_;
        if (shard != idle_scan_shard_idx_) {
            continue; // 跳过
        }
    }

    // 检查该用户的所有连接
    for (auto& conn : kv.second) {
        try {
            auto ctx = std::any_cast<ConnContext>(conn->getContext());
            if (ctx.state != ConnContext::kOpen) continue;

            // 判断空闲超时
            if (now_ms - ctx.last_active_ms > idle_timeout_ms_) {
                to_close.push_back(conn);
                LOG_WARN << "Idle connection detected: user=" << user_id
                    << " idle=" << (now_ms - ctx.last_active_ms) << "ms";
            }
        } catch (const std::bad_any_cast&) {
            continue;
        }
    }
}

// 循环推进分片索引
if (idle_scan_shards_ > 1) {
    idle_scan_shard_idx_ = (idle_scan_shard_idx_ + 1) % idle_scan_shards_;
}

// 锁外关闭连接（避免阻塞）
for (auto& c : to_close) {
    if (c->connected()) {
        c->shutdown();
    }
}
}
}

```

定时触发

```

// comet/comet_server.cpp（构造函数中）

CometServer::CometServer(EventLoop* loop, const Config& cfg)
: server_(loop, ...), loop_(loop) {

    // ...其他初始化...
}

```

```
// 从配置读取参数
idle_timeout_ms_ = static_cast<int64_t>(cfg.comet_idle_timeout_ms); // 默认60000
redis_ttl_ms_ = static_cast<int64_t>(cfg.comet_redis_ttl_ms); // 默认60000

// 注册定时任务（每5秒执行一次）
if (loop_) {
    loop_>runEvery(5.0, std::bind(&CometServer::CheckIdleConnections, this));
}
}
```

2.3.5 Redis TTL兜底

双层保护机制



场景分析

场景	Comet空闲扫描	Redis TTL	结果
客户端正常断开	OnConnection 立即清理	TTL刷新停止，60s后过期	路由立即清除 ✔
客户端僵死（有心跳）	last_active_ms 刷新，不触发	TTL持续刷新	连接保持 ✔
客户端僵死（无心跳）	60s后检测到超时，主动shutdown	TTL刷新停止，60s后过期	约70s内清理 ✔
Comet崩溃	✗ 无法执行	TTL刷新停止，60s后自动清理	依赖Redis兜底 ✔

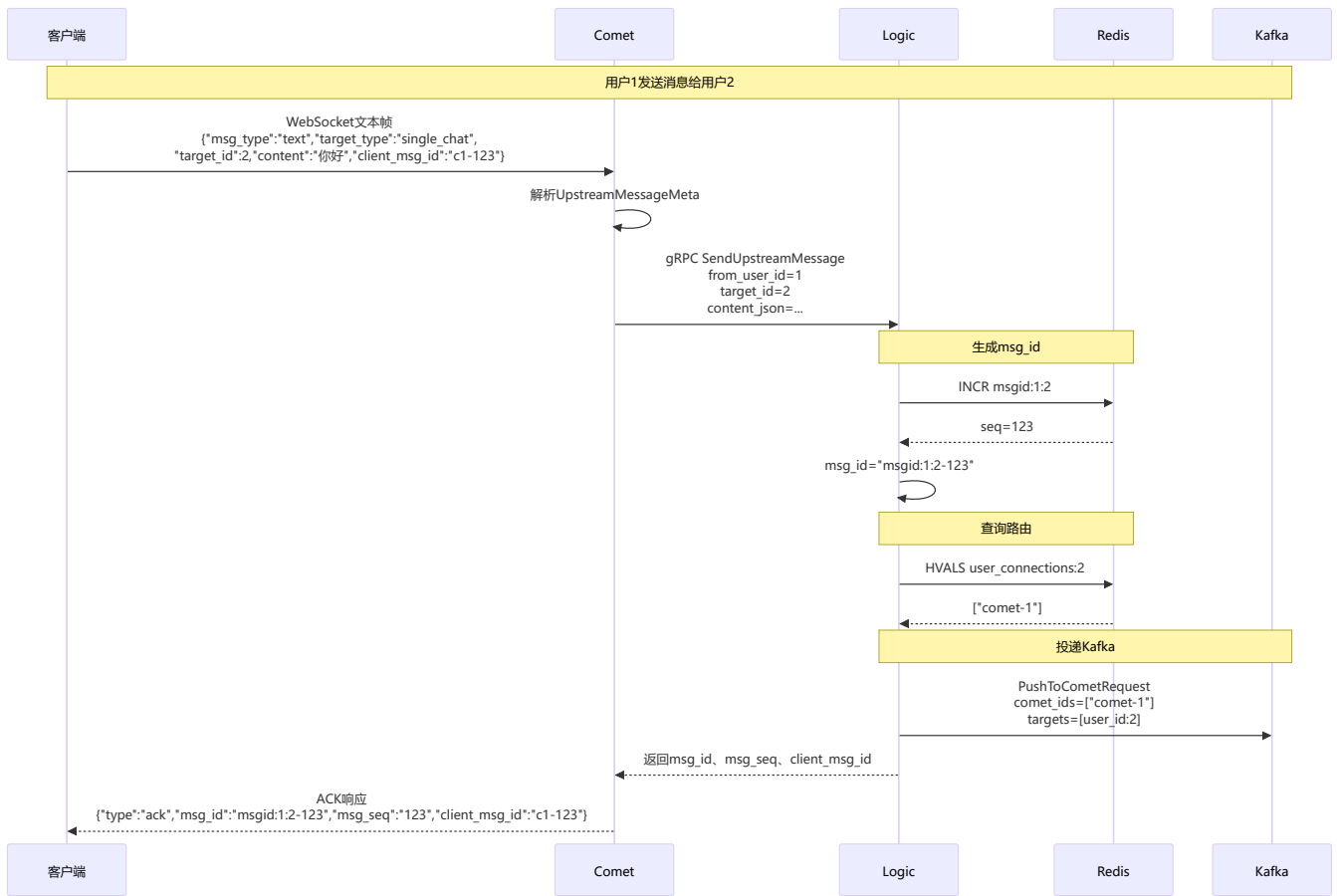
3 消息流转与路由

一对一聊天流程、多连接路由、Kafka集成详解

3.1. 一对一聊天流程

3.1.1 WebSocket上行流程

完整时序图



客户端发送示例

```
# websocat连接（带保活）
websocat "ws://localhost:9000/ws?token=tk-1-xxx" \
  --ping-interval 30

# 发送JSON消息
{"msg_type":"text","target_type":"single_chat","target_id":2,"content":"你好","client_msg_id":"c1-123"}
```

3.1.2 Logic处理流程

SendUpstreamMessage实现

```
// logic/grpc_service.cpp

::grpc::Status LogicServiceImpl::SendUpstreamMessage(
    ::grpc::ServerContext*,
```

```

const UpstreamMessageRequest* request,
UpstreamMessageReply* response) {

// 1. 参数验证
int64_t from_user = request->from_user_id();
int64_t to_user = request->target_id();
if (from_user <= 0 || request->target_type() != "single_chat" || to_user <= 0) {
    return ERROR("invalid parameters");
}

// 2. 构造会话信息
Session session;
session.type = SessionType::kSingle;
session.user1_id = std::min(from_user, to_user);
session.user2_id = std::max(from_user, to_user);
session.id = "s_" + std::to_string(session.user1_id) + ":" +
            std::to_string(session.user2_id);
// 例如: s_1:2

// 3. 生成消息ID (Redis INCR)
int64_t seq = 0;
if (!redis_store->NextSingleMsgId(session.user1_id, session.user2_id, &seq)) {
    return ERROR("generate msg_id failed");
}

std::string msg_id = "msgid:" + std::to_string(session.user1_id) + ":" +
                    std::to_string(session.user2_id) + "-" + std::to_string(seq);
// 例如: msgid:1:2-123

// 4. 补充消息内容 (添加msg_id、msg_seq等字段)
int64_t now_ms = CurrentTimeMillis();
std::string final_json = request->content_json();
try {
    auto j = nlohmann::json::parse(final_json);
    j["msg_id"] = msg_id;
    j["msg_seq"] = seq;
    j["create_time"] = now_ms;
    j["from_user_id"] = from_user;
    j["client_msg_id"] = request->client_msg_id();

    // 查询发送方信息
    User u;
    if (user_dao->GetUserById(from_user, &u, nullptr)) {
        j["sender"] = {{"uid", from_user}, {"name", u.name}};
    }
    final_json = j.dump();
} catch (...) {}

// 5. 构造ChatMessage (用于Kafka投递)
ChatMessage cm;
cm.set_msg_id(msg_id);
cm.set_session_id(session.id);
cm.set_msg_seq(seq);

```



```

// 步骤1: 验证请求方法
if (req.method() != HttpRequest::kPost) {
    WriteJson(resp, 405, "only POST allowed", "{}",
        HttpResponse::k400BadRequest);
    return;
}

// 步骤2: 验证用户身份 (从token获取发送方user_id)
int64_t from_user = 0;
if (!GetUserIdFromRequest(req, &from_user)) {
    WriteJson(resp, 401, "unauthorized");
    return;
}

// 步骤3: 解析JSON请求体
nlohmann::json j;
if (!ParseJsonBody(req.body(), &j)) {
    WriteJson(resp, 400, "invalid json body", "{}",
        HttpResponse::k400BadRequest);
    return;
}

// 步骤4: 提取并验证请求参数
std::string target_type = j.at("target_type").get<std::string>();
int64_t target_id = j.at("target_id").get<int64_t>();
std::string content = j.at("content").get<std::string>();
std::string client_msg_id = j.value("client_msg_id", "");

if (target_type != "single_chat" || target_id <= 0 || content.empty()) {
    WriteJson(resp, 400, "only single_chat supported", "{}",
        HttpResponse::k400BadRequest);
    return;
}

// 步骤5: 生成消息ID和序号
int64_t small_uid = std::min(from_user, target_id);
int64_t large_uid = std::max(from_user, target_id);
std::string session_id =
    "s_" + std::to_string(small_uid) + ":" + std::to_string(large_uid);

std::string msg_id;
int64_t msg_seq = 0;
if (redis_store_ &&
    redis_store_>NextSingleMsgId(small_uid, large_uid, &msg_seq)) {
    msg_id = "msgid:" + std::to_string(small_uid) + ":" +
        std::to_string(large_uid) + "-" + std::to_string(msg_seq);
} else {
    WriteJson(resp, 500, "generate msg_id failed");
    return;
}

// 步骤6: 规范化消息内容JSON并构造ChatMessage
int64_t now_ms = std::chrono::duration_cast<std::chrono::milliseconds>(

```

```

        std::chrono::system_clock::now().time_since_epoch())
        .count();
    nlohmann::json norm;
    norm["msg_id"] = msg_id;
    norm["msg_seq"] = msg_seq;
    norm["create_time"] = now_ms;
    norm["from_user_id"] = from_user;
    norm["target_type"] = "single_chat";
    norm["target_id"] = target_id;
    norm["content"] = content;
    if (!client_msg_id.empty()) norm["client_msg_id"] = client_msg_id;

    ChatMessage cm;
    cm.set_msg_id(msg_id);
    cm.set_session_id(session_id);
    cm.set_msg_seq(msg_seq);
    cm.set_sender_id(from_user);
    cm.set_timestamp_ms(now_ms);
    cm.set_content_json(norm.dump());
    cm.set_client_msg_id(client_msg_id);

    // 步骤7: 调用后置处理推送消息
    PostProcessSingleMessage(target_id, session_id, cm, redis_store_,
                             push_producer_);

    // 步骤8: 返回成功响应
    std::ostringstream data;
    data << "{"
        << "\"msg_id\": \"" << msg_id << "\", "
        << "\"msg_seq\": " << msg_seq << ", "
        << "\"session_id\": \"" << session_id << "\" "
        << "}";
    writeJson(resp, 0, "ok", data.str());
}

```

3.2. 多连接路由设计

3.2.1 Redis路由结构

HASH结构设计

```

Key:    user_connections:{user_id}
Type:   HASH
Field:  {conn_id}
Value:  {comet_id}
TTL:    PEXPIRE 60000ms (心跳刷新)

```

实际示例

```
# 用户1在 comet-1 上同时存在 2 个连接（系统不区分 device，只区分连接）
127.0.0.1:6379> HGETALL user_connections:1
1) "comet-1-1734567890-1"
2) "comet-1"
3) "comet-1-1734567900-2"
4) "comet-1"

# 用户2跨 Comet 登录（两个连接分别落在不同 comet 上）
127.0.0.1:6379> HGETALL user_connections:2
1) "comet-1-1734567910-3"
2) "comet-1"
3) "comet-2-1734567920-1"
4) "comet-2"

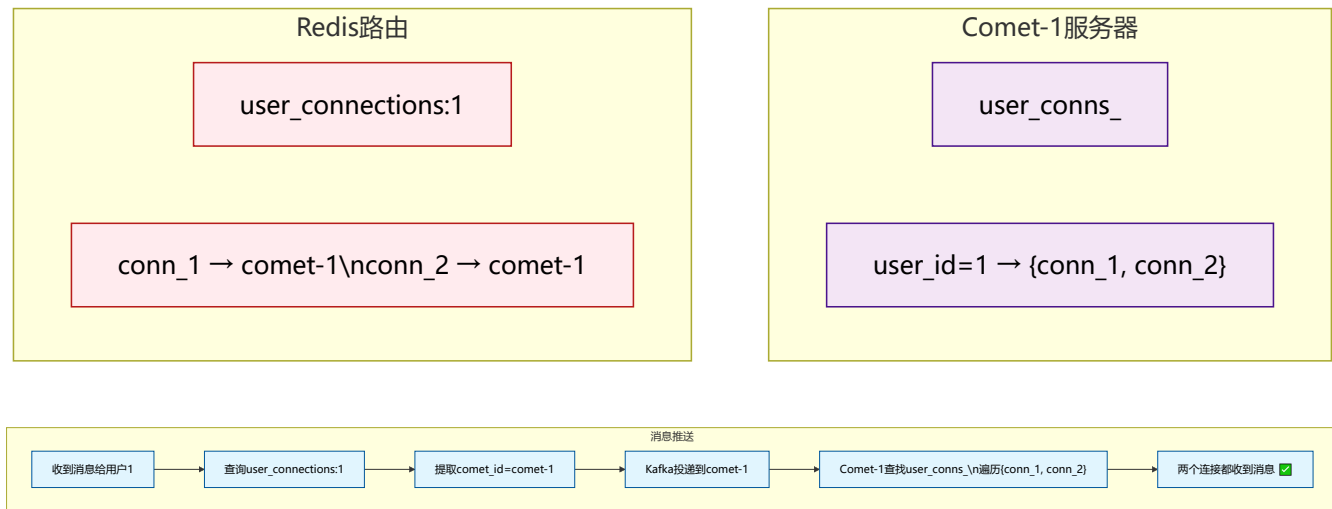
# 提取comet_id列表（应用层去重）
# user_connections:1 → ["comet-1"]（去重后）
# user_connections:2 → ["comet-1", "comet-2"]（去重后）
```

⚠ 注意：conn_id 通常由 comet 生成，可能“看起来”带有 comet_id 前缀（便于排障），但它依然只是连接标识，并不代表 device。

如果你的 Redis 里还混入了类似 "comet-1:<conn_id>" -> "{}" 的额外 field/value（旧逻辑/外部系统写入），请避免与路由数据写在同一个 user_connections:* Hash 中；本项目路由查询走的是 HVALS 取 value 作为 comet_id 列表，混入非 comet_id 的 value 会污染路由结果。

3.2.2 一用户多连接场景

场景：同一用户同时存在多个连接（不区分device）



Comet内存结构

```
// comet/comet_server.h

// 用户连接映射: user_id → 该用户在本进程内的连接集合
std::unordered_map<int64_t, std::set<TcpConnectionPtr>> user_conns_;

// 示例数据:
// user_conns_[1] = {conn_1, conn_2}
// user_conns_[2] = {conn_1}
```

推送到所有连接

```
// comet/comet_server.cpp

void CometServer::PushToUsers(
    const ChatMessage& msg,
    const std::vector<int64_t>& user_ids) {

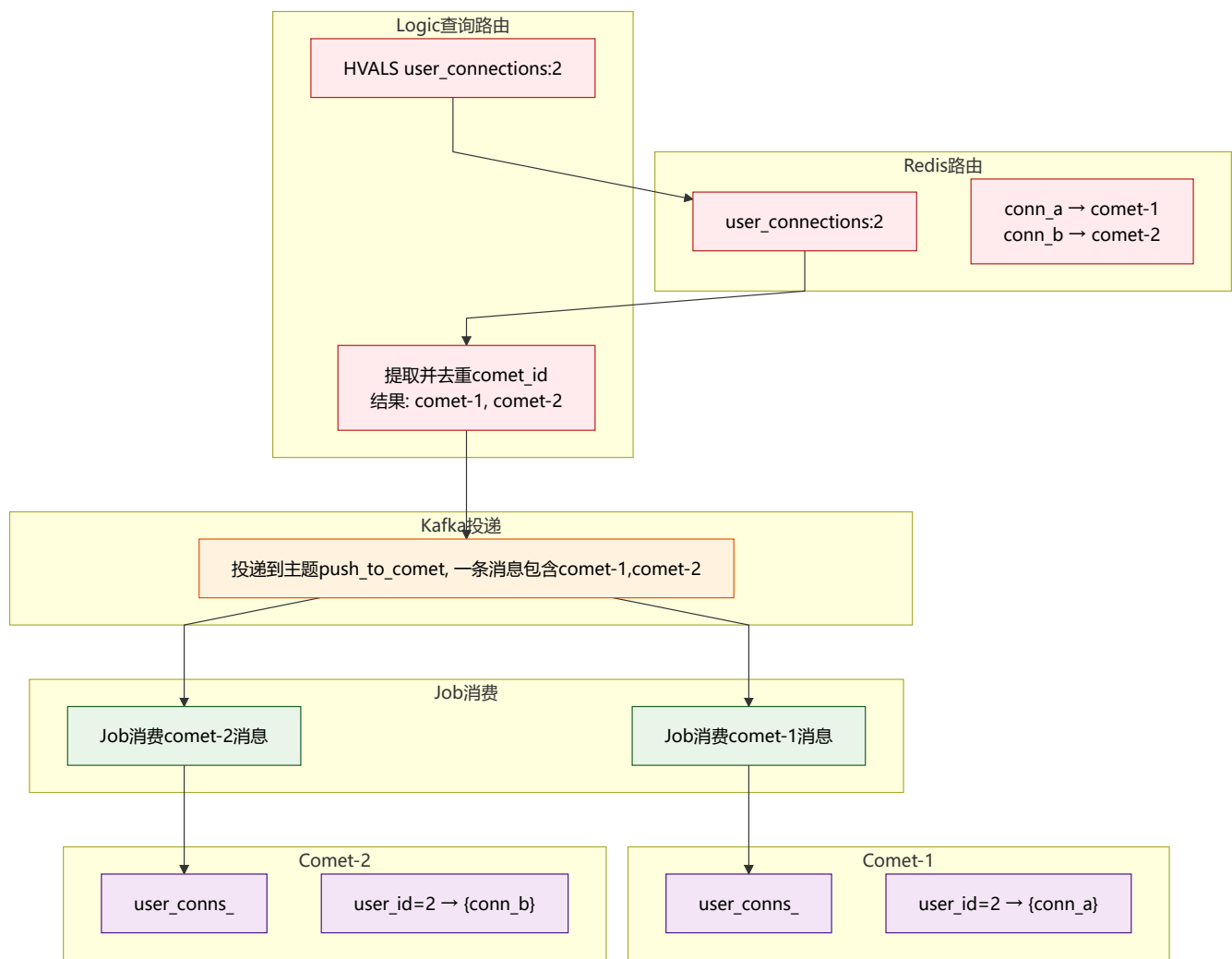
    // 1. 构造WebSocket文本帧
    std::string payload = msg.content_json();
    std::string frame = BuildWebSocketTextFrame(payload);

    // 2. 查找所有目标用户的所有连接
    std::vector<TcpConnectionPtr> conns;
    {
        std::lock_guard<std::mutex> lock(conns_mu_);
        for (int64_t uid : user_ids) {
            auto it = user_conns_.find(uid);
            if (it != user_conns_.end()) {
                // 将该用户的所有连接都加入推送列表
                for (const auto& c : it->second) {
                    conns.push_back(c);
                }
            }
        }
    }

    // 3. 批量发送到所有连接（多连接同时推送）
    for (const auto& c : conns) {
        if (c->connected()) {
            c->send(frame); // 每个连接都会收到消息 ✅
        }
    }
}
```

3.2.3 跨Comet路由

场景：同一用户的不同连接分别连接到不同Comet



GetUserConnectionComets实现（从实际源码）

```
// logic/redis_store.cpp

bool RedisStore::GetUserConnectionComets(int64_t user_id,
                                         std::vector<std::string>* comets) {

    // 步骤1: 参数验证
    if (!pool_ || !comets) return false;

    // 步骤2: 获取 Redis 连接
    auto guard = pool_>Acquire();
    redisContext* ctx = guard.get();
    if (!ctx) return false;

    // 步骤3: 构造 Redis key
    std::string key = "user_connections:" + std::to_string(user_id);

    // 步骤4: 执行 HVALS 命令（返回所有value, 即comet_id）
    redisReply* reply = (redisReply*)redisCommand(ctx, "HVALS %s", key.c_str());
```

```
if (!reply) {
    LOG_ERROR << "Redis HVALS user_connections failed";
    return false;
}

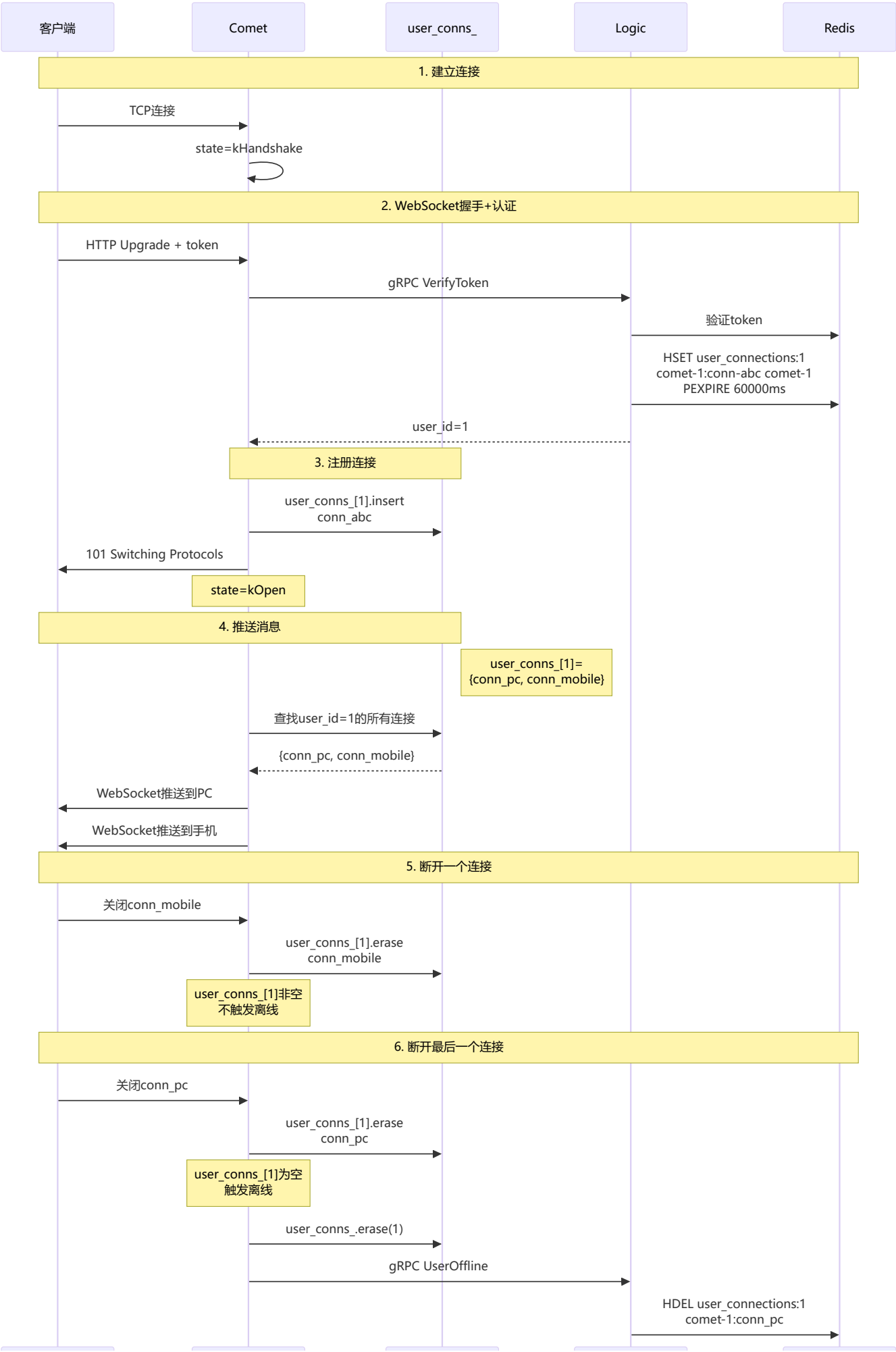
// 步骤5: 解析返回结果并去重
bool ok = false;
if (reply->type == REDIS_REPLY_ARRAY) {
    ok = true;
    // 使用 unordered_set 去重 (同一comet可能有多个连接)
    std::unordered_set<std::string> uniq;

    // 遍历数组中的每个元素 (comet_id)
    for (size_t i = 0; i < reply->elements; ++i) {
        redisReply* v = reply->element[i];
        if (v->type == REDIS_REPLY_STRING && v->str && v->len > 0) {
            uniq.insert(v->str); // 自动去重
        }
    }

    // 将去重后的结果赋值给输出参数
    comets->assign(uniq.begin(), uniq.end());
}

freeReplyObject(reply);
return ok;
}
```

3.2.4 连接生命周期



辅助函数：PostProcessSingleMessage

```
// logic/message_helper.cpp (实际源码)

bool PostProcessSingleMessage(int64_t target_user,
                              const std::string& session_id,
                              const ChatMessage& cm, RedisStore* redis_store,
                              kafkaProducer* push_producer) {

    // 步骤1: 验证 Redis 存储对象是否可用
    // 如果为空, 无法查询用户连接信息, 直接返回失败
    if (!redis_store) {
        LOG_ERROR << "redis_store is null in PostProcessSingleMessage";
        return false;
    }

    // 步骤2: 验证 Kafka 生产者是否可用
    // 如果为空, 无法发送推送消息到 comet, 直接返回失败
    if (!push_producer) {
        LOG_ERROR << "kafka producer not ready for single message session="
                  << session_id;
        return false;
    }

    // 步骤3: 构造推送请求对象
    // PushToCometRequest 是 protobuf 定义的消息体, 包含消息内容和目标信息
    PushToCometRequest req;
    // 将聊天消息复制到推送请求中
    *req.mutable_message() = cm;

    // 步骤4: 从 Redis 查询目标用户当前连接的所有 comet 节点
    // 一个用户可能在多个设备/浏览器上登录, 因此可能有多个 comet 连接
    std::vector<std::string> comets;
    if (redis_store->GetUserConnectionComets(target_user, &comets)) {
        // 遍历所有 comet 节点, 为每个节点添加推送目标
        for (const auto& cid : comets) {
            // 添加 comet_id 到推送请求的 comet_ids 列表
            req.add_comet_ids(cid);
            // 添加推送目标, 包含用户 ID 和对应的 comet_id
            auto* t = req.add_targets();
            t->set_user_id(target_user);
            t->set_comet_id(cid);
        }
    }

    // 步骤5: 处理用户离线的情况
    // 如果用户没有在线连接 (comets 为空), 仍然构造一个推送目标
    // 这样可以让 comet 节点进行离线推送处理 (如离线消息存储、推送通知等)
    if (comets.empty()) {
```

```

        auto* t = req.add_targets();
        t->set_user_id(target_user);
        LOG_WARN << "No online target for message " << session_id
                  << ", comet_ids empty";
    }

    // 步骤6: 将推送请求序列化为字节流
    // protobuf 的 SerializeToString 将消息对象转换为二进制格式
    std::string payload;
    if (!req.SerializeToString(&payload)) {
        LOG_ERROR << "Serialize PushToCometRequest failed for session "
                  << session_id;
        return false;
    }

    // 步骤7: 通过 kafka 发送推送请求
    // 使用 session_id 作为 kafka 的 key, 便于追踪和调试
    // Send 方法是异步的, 返回 true 表示提交成功, 不代表一定发送成功
    LOG_INFO << "Send payload for session " << session_id;
    if (!push_producer->Send(session_id, payload)) {
        LOG_ERROR << "kafka send failed for session " << session_id;
        return false;
    }

    // 返回成功, 表示消息已提交到 kafka
    return true;
}

```

3.3. Kafka消息队列

3.3.1 Producer配置与投递

Producer初始化 (从实际源码)

```

// common/kafka_producer.cpp

bool KafkaProducer::Init(const std::string& brokers, const std::string& topic) {
    std::string errstr;
    topic_name_ = topic;

    RdKafka::Conf* conf = RdKafka::Conf::create(RdKafka::Conf::CONF_GLOBAL);

    // 基础配置
    if (!set_conf("bootstrap.servers", brokers)) {
        delete conf;
        return false;
    }

    // 可靠性配置
    set_conf("acks", "all");
    set_conf("enable.idempotence", "true");
}

```

```

// 压缩配置（尝试lz4/snappy/gzip，失败则不压缩）
bool compression_ok =
    set_conf_any({"compression.type", "lz4"}, {"compression.codec", "lz4"}) ||
    set_conf_any({"compression.type", "snappy"}, {"compression.codec", "snappy"}) ||
    set_conf_any({"compression.type", "gzip"}, {"compression.codec", "gzip"});
if (!compression_ok) {
    set_conf_with_values("compression.type", {"none"});
    LOG_WARN << "KafkaProducer compression unavailable, fallback to none";
}

// 性能优化
set_conf("linger.ms", "5");
set_conf("batch.num.messages", "1000");
set_conf("retries", "5");
set_conf("retry.backoff.ms", "500");
set_conf("message.timeout.ms", "60000");

// 创建Producer
producer_.reset(RdKafka::Producer::create(conf, errstr));
delete conf;
if (!producer_) {
    LOG_ERROR << "KafkaProducer set brokers failed: " << errstr;
    return false;
}

// 绑定topic
RdKafka::Conf* topic_conf = RdKafka::Conf::create(RdKafka::Conf::CONF_TOPIC);
topic_.reset(
    RdKafka::Topic::create(producer_.get(), topic_name_, topic_conf, errstr));
delete topic_conf;
if (!topic_) {
    LOG_ERROR << "KafkaProducer create topic failed: " << errstr;
    return false;
}

LOG_INFO << "KafkaProducer initialized topic=" << topic_name_;
return true;
}

```

发送消息（从实际源码）

```

// common/kafka_producer.cpp

bool KafkaProducer::Send(const std::string& key, const std::string& value) {
    if (!producer_ || !topic_) {
        return false;
    }

    RdKafka::ErrorCode err = producer_>produce(
        topic_.get(),
        RdKafka::Topic::PARTITION_UA, // 根据key自动选择分区

```

```

        RdKafka::Producer::RK_MSG_COPY,
        const_cast<char*>(value.data()),
        value.size(),
        key.empty() ? nullptr : &key,    // 分区键
        nullptr
    );

    if (err != RdKafka::ERR_NO_ERROR) {
        LOG_ERROR << "kafka produce failed: " << RdKafka::err2str(err);
        return false;
    }

    // poll 触发内部回调与发送进度
    producer_>poll(0);
    return true;
}

```

3.3.2 Consumer配置与消费

Consumer初始化（从实际源码）

```

// common/kafka_consumer.cpp

bool KafkaConsumer::Init(const std::string& brokers,
                        const std::string& group_id,
                        const std::string& topic,
                        std::function<void(const std::string&, const std::string&)>
callback,
                        const Options& options) {
    topic_ = topic;
    callback_ = std::move(callback);
    options_ = options;

    std::string errstr;
    RdKafka::Conf* conf = RdKafka::Conf::create(RdKafka::Conf::CONF_GLOBAL);
    conf->set("bootstrap.servers", brokers, errstr);
    conf->set("group.id", group_id, errstr);
    conf->set("enable.partition.eof", "false", errstr);
    conf->set("auto.offset.reset", options_.auto_offset_reset, errstr);
    conf->set("enable.auto.commit",
            options_.enable_auto_commit ? "true" : "false",
            errstr);
    conf->set("auto.commit.interval.ms",
            std::to_string(options_.auto_commit_interval_ms),
            errstr);
    conf->set("max.poll.interval.ms",
            std::to_string(options_.max_poll_interval_ms),
            errstr);
    conf->set("session.timeout.ms",
            std::to_string(options_.session_timeout_ms),
            errstr);
}

```

```

consumer_.reset(RdKafka::KafkaConsumer::create(conf, errstr));
delete conf;
if (!consumer_) {
    LOG_ERROR << "KafkaConsumer create failed: " << errstr;
    return false;
}

std::vector<std::string> topics = {topic_};
RdKafka::ErrorCode err = consumer_>subscribe(topics);
if (err != RdKafka::ERR_NO_ERROR) {
    LOG_ERROR << "KafkaConsumer subscribe failed: " << RdKafka::err2str(err);
    return false;
}
return true;
}

```

消费循环 (从实际源码)

```

// common/kafka_consumer.cpp

void KafkaConsumer::Loop() {
    while (running_) {
        std::unique_ptr<RdKafka::Message> msg(consumer_>consume(1000));
        if (!msg) {
            continue;
        }
        HandleMessage(msg.get());
    }
}

void KafkaConsumer::HandleMessage(RdKafka::Message* message) {
    switch (message->err()) {
        case RdKafka::ERR_NO_ERROR: {
            std::string key;
            if (message->key()) {
                key = *message->key();
            }
            std::string value(static_cast<const char*>(message->payload()),
                             message->len());
            if (callback_) {
                callback_(key, value);
            }
            if (!options_.enable_auto_commit && consumer_) {
                consumer_>commitAsync(message);
            }
            break;
        }
        case RdKafka::ERR__TIMED_OUT:
            break;
        default:
            LOG_ERROR << "Kafka consume error: " << message->errstr();
    }
}

```

```
        break;
    }
}
```

3.3.3 Job处理流程

HandleMessage实现

```
// job/service.cpp

void JobRunner::HandleMessage(const std::string& key, const std::string& value) {
    (void)key; // 当前未使用 kafka 消息的 key

    // 提交到线程池异步处理（避免阻塞kafka消费线程）
    rpc_pool_.Submit([this, payload = value]() {
        PushToCometRequest req;
        if (!req.ParseFromString(payload)) {
            LOG_ERROR << "Failed to parse PushToCometRequest from kafka";
            return;
        }
        ProcessPushRequest(req);
    });
}

void JobRunner::ProcessPushRequest(const PushToCometRequest& req) {
    // 1. 确定目标 Comet 节点列表
    std::vector<std::string> comet_ids;
    if (req.comet_ids_size() > 0) {
        // 使用请求中指定的 Comet 节点
        comet_ids.reserve(req.comet_ids_size());
        for (const auto& cid : req.comet_ids()) {
            comet_ids.push_back(cid);
        }
    } else {
        // 未指定时，广播到所有已配置的 Comet 节点
        for (const auto& kv : comet_addrs_) {
            comet_ids.push_back(kv.first);
        }
    }

    // 2. 检查是否有可用的 Comet 节点
    if (comet_ids.empty()) {
        LOG_ERROR << "No comet targets available, drop msg_id="
            << req.message().msg_id();
        return;
    }

    // 3. 遍历目标 Comet 节点，依次发送推送请求
    for (const auto& cid : comet_ids) {
        // 3.1 获取该 Comet 节点的 gRPC Stub
        CometService::Stub* stub = GetStub(cid);
    }
}
```

```

if (!stub) {
    LOG_ERROR << "Unknown comet_id " << cid
        << " for msg_id=" << req.message().msg_id();
    continue; // Stub 获取失败，跳过该节点
}

// 3.2 构造发送给单个 Comet 的请求（关键：按 comet_id 过滤 targets）
//
// 说明：
// - logic/message_helper 会为同一 user_id 在不同 comet_id 上生成多个 targets
// - 如果 job 不按 cid 过滤 targets，会导致每个 comet 收到包含“其他 comet”的 targets
// 从而在 comet 端按 user_id 推送时出现重复投递
PushToCometRequest req2;
*req2.mutable_message() = req.message();
req2.add_comet_ids(cid); // 便于 Comet 端日志记录

// 如果 target.comet_id 为空，认为是“未绑定节点”的目标（兼容旧逻辑/离线场景）
// 否则只保留 target.comet_id == cid 的目标
for (const auto& t : req.targets()) {
    if (t.comet_id().empty() || t.comet_id() == cid) {
        *req2.add_targets() = t;
    }
}

// 若过滤后没有目标用户，直接跳过，避免无意义 RPC
if (req.targets_size() > 0 && req2.targets_size() == 0) {
    LOG_INFO << "Skip PushToComet to comet_id=" << cid
        << " because no matching targets. msg_id="
        << req.message().msg_id();
    continue;
}

// 3.3 发起 gRPC 调用
PushToCometReply reply;
grpc::ClientContext ctx;
LOG_INFO << "PushToComet to comet_id=" << cid
    << ", msg_id=" << req.message().msg_id();
auto status = stub->PushToComet(&ctx, req2, &reply);

// 3.4 检查 RPC 调用是否成功
if (!status.ok()) {
    LOG_ERROR << "PushToComet RPC failed to comet_id=" << cid << ": "
        << status.error_message();
    continue;
}

// 3.5 检查业务层返回的错误码
if (reply.error().code() != 0) {
    LOG_ERROR << "Comet response error to comet_id=" << cid << ": "
        << reply.error().message();
}
}
}

```

3.4. Comet下行推送

4.1 PushToComet接口（从实际源码）

```
// comet/comet_grpc_service.cpp

::grpc::Status CometServiceImpl::PushToComet(
    ::grpc::ServerContext *, const ::sparkpush::PushToCometRequest *request,
    ::sparkpush::PushToCometReply *response) {

    // 提取comet_id（从comet_ids列表中取第一个）
    std::string comet_id =
        request->comet_ids_size() > 0 ? request->comet_ids(0) : "";

    // 记录日志
    LOG_INFO << "PushToComet received for comet_id=" << comet_id
        << " msg_id=" << request->message().msg_id();

    // 策略1: 精确推送（targets列表非空）
    if (request->targets_size() > 0) {
        // 提取所有目标用户的user_id
        std::vector<int64_t> users; // 目标用户 ID 列表（去重后）
        users.reserve(request->targets_size()); // 预分配内存，避免多次扩容
        std::unordered_set<int64_t> seen;
        seen.reserve(static_cast<size_t>(request->targets_size()));

        for (const auto &t : request->targets()) {
            LOG_INFO << "PushToComet target user_id=" << t.user_id();
            if (seen.insert(t.user_id()).second) {
                users.push_back(t.user_id());
            }
        }

        // 调用CometServer::PushToUsers执行推送
        server_->PushToUsers(request->message(), users);
    }

    // 策略2: 广播推送（根据session_id判断）
    else {
        const std::string &sid = request->message().session_id();

        if (sid == "broadcast") {
            // 全服广播
            server_->PushBroadcast(request->message());
        } else {
            LOG_WARN << "PushToComet targets empty but unsupported session_id="
                << sid;
        }
    }

    // 返回成功响应
    response->mutable_error()->set_code(0);
}
```

```
response->mutable_error()->set_message("ok");
return ::grpc::Status::OK;
}
```

4.2 多设备推送（从实际源码）

```
// comet/comet_server.cpp

void CometServer::PushToUsers(const ChatMessage &msg,
                              const std::vector<int64_t> &user_ids) {
    // 步骤1: 构造WebSocket文本帧
    std::string payload = msg.content_json().empty()
        ? ("{\"msg_id\":\"" + msg.msg_id() + "\"}")
        : msg.content_json();
    std::string frame = BuildWebSocketTextFrame(payload);

    // 步骤2: 查找目标连接（需要加锁访问user_conns_）
    std::vector<TcpConnectionPtr> conns;
    std::vector<std::pair<int64_t, TcpConnectionPtr>> stale;

    {
        std::lock_guard<std::mutex> lock(conns_mu_);

        // 遍历所有目标用户ID
        for (int64_t uid : user_ids) {
            auto it = user_conns_.find(uid);
            if (it == user_conns_.end()) continue;

            // 遍历该用户的所有连接（多端登录）
            for (const auto &c : it->second) {
                if (c && c->connected()) {
                    conns.push_back(c);
                } else {
                    stale.emplace_back(uid, c);
                }
            }
        }

        // 清理失效连接
        for (const auto &rm : stale) {
            it->second.erase(rm.second);
        }
        stale.clear();

        // 如果该用户的所有连接都已清理，删除整个映射项
        if (it != user_conns_.end() && it->second.empty()) {
            user_conns_.erase(it);
        }
    }

    // 步骤3: 发送消息（在锁外执行，避免持锁进行I/O）
```

```
    for (const auto &c : conns) {
        if (c->connected()) {
            c->send(frame); // 真正发给web客户端
        }
    }
}
```

4 服务架构详解

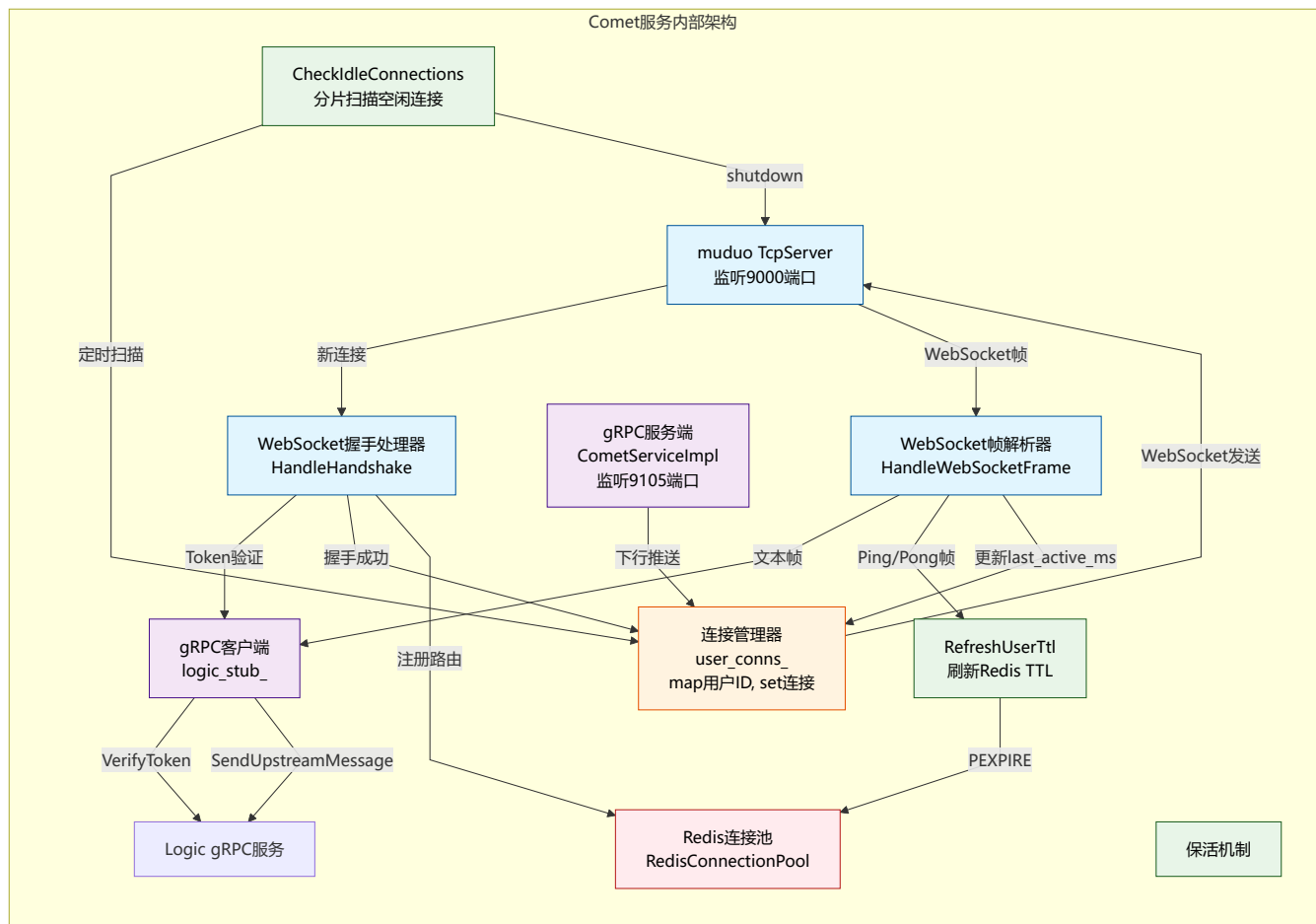
Comet、Logic、Job三大服务内部实现剖析

4.1. Comet服务架构

4.1.1 核心组件

组件	类型	职责
muduo TcpServer	网络库	监听WebSocket连接（端口9000）
gRPC Server	RPC服务端	接收Job的下行推送（端口9105）
gRPC Client	RPC客户端	调用Logic的VerifyToken、SendUpstreamMessage
user_conns_	内存映射	map<user_id, set<conn>> 维护用户连接
Redis连接池	缓存客户端	路由注册、TTL刷新
保活机制	定时任务	RefreshUserTtl、CheckIdleConnections

4.1.2 comet架构图



4.1.3 关键实现

服务启动

```
// comet/main.cpp

int main(int argc, char* argv[]) {
    // 1. 加载配置
    Config cfg;
    if (!LoadConfig(argv[1], &cfg)) {
        return 1;
    }

    // 2. 创建muduo事件循环
    muduo::net::EventLoop loop;

    // 3. 创建Comet服务器
    CometServer server(&loop, cfg);
    server.SetThreadNum(cfg.comet_io_threads); // IO线程数 (默认8)
    server.Start();

    // 4. 创建gRPC服务端 (接收Job推送)
```

```

std::string grpc_addr = "0.0.0.0:" + std::to_string(cfg.comet_grpc_port);
grpc::ServerBuilder builder;
builder.AddListeningPort(grpc_addr, grpc::InsecureServerCredentials());

CometServiceImpl grpc_service(&server);
builder.RegisterService(&grpc_service);

std::unique_ptr<grpc::Server> grpc_server(builder.BuildAndStart());
LOG_INFO << "Comet gRPC server listening on " << grpc_addr;

// 5. gRPC服务在独立线程运行
std::thread grpc_thread([&grpc_server]() {
    grpc_server->wait();
});

// 6. muduo事件循环在主线程运行
loop.loop(); // 阻塞，直到loop.quit()

grpc_server->Shutdown();
grpc_thread.join();
return 0;
}

```

连接管理核心数据结构

```

// comet/comet_server.h

class CometServer {
private:
    // 用户连接映射: user_id → 该用户在本进程内的连接集合
    // - 一个用户可能多端登录，因此使用set存储多个连接
    // - 用于下行推送时按user_id找到对应连接集合
    std::unordered_map<int64_t, std::set<TcpConnectionPtr>> user_conns_;

    // 保护user_conns_的互斥锁（多线程IO模式下使用）
    mutable std::mutex conns_mu_;
};

```

握手成功后注册连接

```

// comet/comet_server.cpp

void CometServer::HandleHandshake(...) {
    // ...验证token...
    int64_t user_id = vrep.user_id();

    // 1. 完成WebSocket握手
    conn->send("HTTP/1.1 101 Switching Protocols\r\n...");

    // 2. 更新连接状态
    ConnContext ctx = std::any_cast<ConnContext>(conn->getContext());
    ctx.state = ConnContext::kOpen;
}

```

```

ctx.user_id = user_id;
ctx.last_active_ms = CurrentTimeMillis();
conn->setContext(ctx);

// 3. 将连接注册到user_conns_ (关键步骤)
{
    std::lock_guard<std::mutex> lock(conns_mu_);
    user_conns_[user_id].insert(conn); // 插入到该用户的连接集合
}

LOG_INFO << "WebSocket handshake done, user_id=" << user_id;
}

```

连接断开清理

```

void CometServer::OnConnection(const TcpConnectionPtr& conn) {
    if (!conn->connected()) {
        // 连接断开: 清理逻辑
        int64_t offline_uid = 0;
        bool need_offline = false;

        try {
            auto ctx = std::any_cast<ConnContext>(conn->getContext());
            if (ctx.user_id > 0) {
                std::lock_guard<std::mutex> lock(conns_mu_);
                auto it = user_conns_.find(ctx.user_id);
                if (it != user_conns_.end()) {
                    // 从该用户的连接集合中移除
                    it->second.erase(conn);

                    // 如果该用户没有任何连接了, 标记为离线
                    if (it->second.empty()) {
                        user_conns_.erase(it);
                        offline_uid = ctx.user_id;
                        need_offline = true;
                    }
                }
            }
        } catch (const std::bad_any_cast&) {}

        // 通知Logic用户离线 (只有所有连接都断开时才通知)
        if (need_offline && offline_uid > 0) {
            NotifyUserOffline(offline_uid);
        }
    }
}

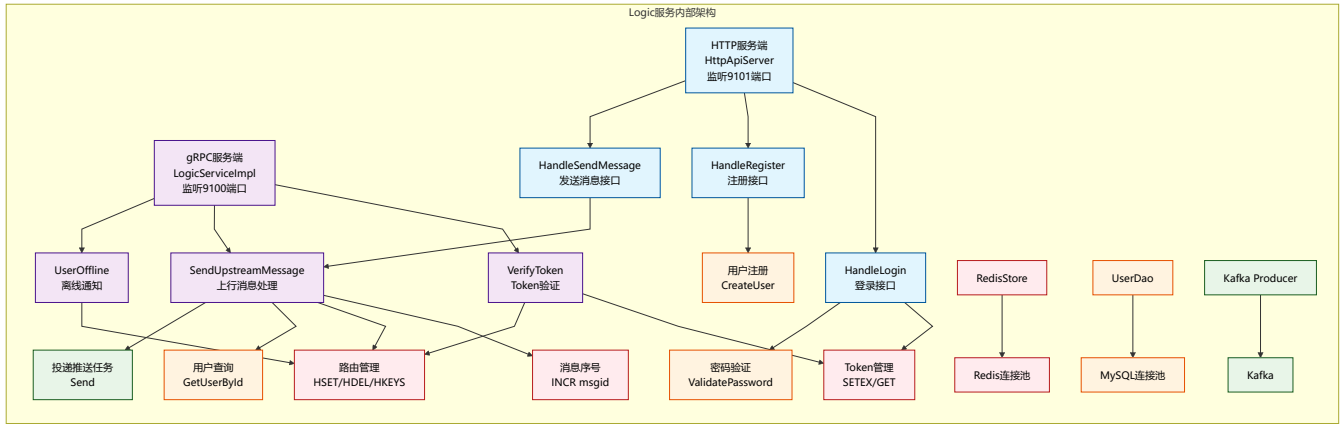
```

4.2. Logic服务架构

4.2.1 核心组件

组件	类型	职责
gRPC Server	RPC服务端	提供VerifyToken、SendUpstreamMessage、UserOffline接口（端口9100）
HTTP Server	HTTP服务端	提供注册、登录、发送消息、查询会话等REST API（端口9101）
RedisStore	缓存封装	Token管理、路由查询、消息序号生成、会话管理
UserDao	数据库封装	用户数据CRUD（MySQL）
KafkaProducer	消息队列	推送任务投递到Kafka
MessageHelper	业务逻辑	后置处理：查询路由并投递Kafka

4.2.2 logic架构图



4.2.3 关键实现

服务启动

```
// logic/main.cpp

int main(int argc, char* argv[]) {
    // 1. 加载配置
    Config cfg;
    if (!LoadConfig(argv[1], &cfg)) {
        return 1;
    }

    // 2. 初始化Redis连接池
    RedisConnectionPool redis_pool;
```

```

redis_pool.Init(cfg.redis_host, cfg.redis_port,
                cfg.redis_password, cfg.redis_db,
                cfg.redis_pool_size);
RedisStore redis_store(&redis_pool);

// 3. 初始化MySQL连接池
MySQLConnectionPool mysql_pool;
mysql_pool.Init(cfg.mysql_host, cfg.mysql_port,
                cfg.mysql_user, cfg.mysql_password,
                cfg.mysql_db, cfg.mysql_pool_size);
UserDao user_dao(&mysql_pool);

// 4. 初始化Kafka Producer
KafkaProducer producer;
producer.Init(cfg.kafka_brokers, cfg.kafka_push_topic);

// 5. 创建gRPC服务端
std::string grpc_addr = "0.0.0.0:" + std::to_string(cfg.listen_port);
grpc::ServerBuilder builder;
builder.AddListeningPort(grpc_addr, grpc::InsecureServerCredentials());

LogicServiceImpl grpc_service(&user_dao, &producer, &redis_store, nullptr);
builder.RegisterService(&grpc_service);

std::unique_ptr<grpc::Server> grpc_server(builder.BuildAndStart());
LOG_INFO << "Logic gRPC server started on " << grpc_addr;

// 6. gRPC服务在独立线程运行
std::thread grpc_thread([&grpc_server]() {
    grpc_server->Wait();
});

// 7. 创建HTTP服务器（在主线程运行）
muduo::net::EventLoop loop;
muduo::net::InetAddress http_addr("0.0.0.0", cfg.http_port);
HttpApiServer http_server(&loop, http_addr, &user_dao, &redis_store, &grpc_service);
http_server.Start();

LOG_INFO << "HTTP server started on 0.0.0.0:" << cfg.http_port;

// 8. muduo事件循环在主线程运行
loop.loop();

grpc_server->Shutdown();
grpc_thread.join();
return 0;
}

```

RedisStore核心方法

```
// logic/redis_store.cpp

// Token管理
bool RedisStore::SetToken(const std::string& token, int64_t user_id, int ttl_seconds) {
    auto guard = pool_>Acquire(0);
    redisContext* ctx = guard.get();
    if (!ctx) return false;

    std::string key = "token:" + token;
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "SETEX %s %d %ld", key.c_str(), ttl_seconds, user_id);

    bool ok = (reply && reply->type == REDIS_REPLY_STATUS);
    if (reply) freeReplyObject(reply);
    return ok;
}

// 消息序号生成（从实际源码）
bool RedisStore::NextSingleMsgId(int64_t user1, int64_t user2, int64_t* seq) {
    if (!seq) return false;
    if (user1 <= 0 || user2 <= 0) return false;

    // 规范化用户ID顺序
    if (user1 > user2) std::swap(user1, user2);

    if (!pool_) return false;

    auto guard = pool_>Acquire();
    redisContext* ctx = guard.get();
    if (!ctx) return false;

    // 构造Redis key: msgid:<小uid>:<大uid>
    std::string key =
        "msgid:" + std::to_string(user1) + ":" + std::to_string(user2);

    // 执行INCR命令
    redisReply* reply = (redisReply*)redisCommand(ctx, "INCR %s", key.c_str());
    if (!reply) {
        LOG_ERROR << "Redis INCR single msg_id failed, key: " << key;
        return false;
    }

    bool ok = false;
    if (reply->type == REDIS_REPLY_INTEGER) {
        ok = true;
        *seq = reply->integer;
    }

    freeReplyObject(reply);
    return ok;
}
```

```

// 路由管理（从实际源码）
bool RedisStore::UpsertUserConnection(int64_t user_id,
                                       const std::string& comet_id,
                                       const std::string& conn_id,
                                       int64_t ttl_ms) {

    // 参数验证
    if (!pool_ || comet_id.empty() || conn_id.empty()) return false;

    auto guard = pool_>Acquire();
    redisContext* ctx = guard.get();
    if (!ctx) return false;

    // 构造Redis key和field
    std::string key = "user_connections:" + std::to_string(user_id);
    std::string field = conn_id; // field使用conn_id

    // 执行HSET命令
    redisReply* reply = (redisReply*)redisCommand(
        ctx, "HSET %s %s %s", key.c_str(), field.c_str(), comet_id.c_str());
    if (!reply) {
        LOG_ERROR << "Redis HSET user_connections failed";
        return false;
    }

    bool ok = (reply->type != REDIS_REPLY_ERROR);
    freeReplyObject(reply);
    if (!ok) return false;

    // 设置TTL
    if (ttl_ms > 0) {
        reply = (redisReply*)redisCommand(ctx, "PEXPIRE %s %lld", key.c_str(),
                                           static_cast<long long>(ttl_ms));
        if (reply) freeReplyObject(reply);
    }
    return true;
}

```

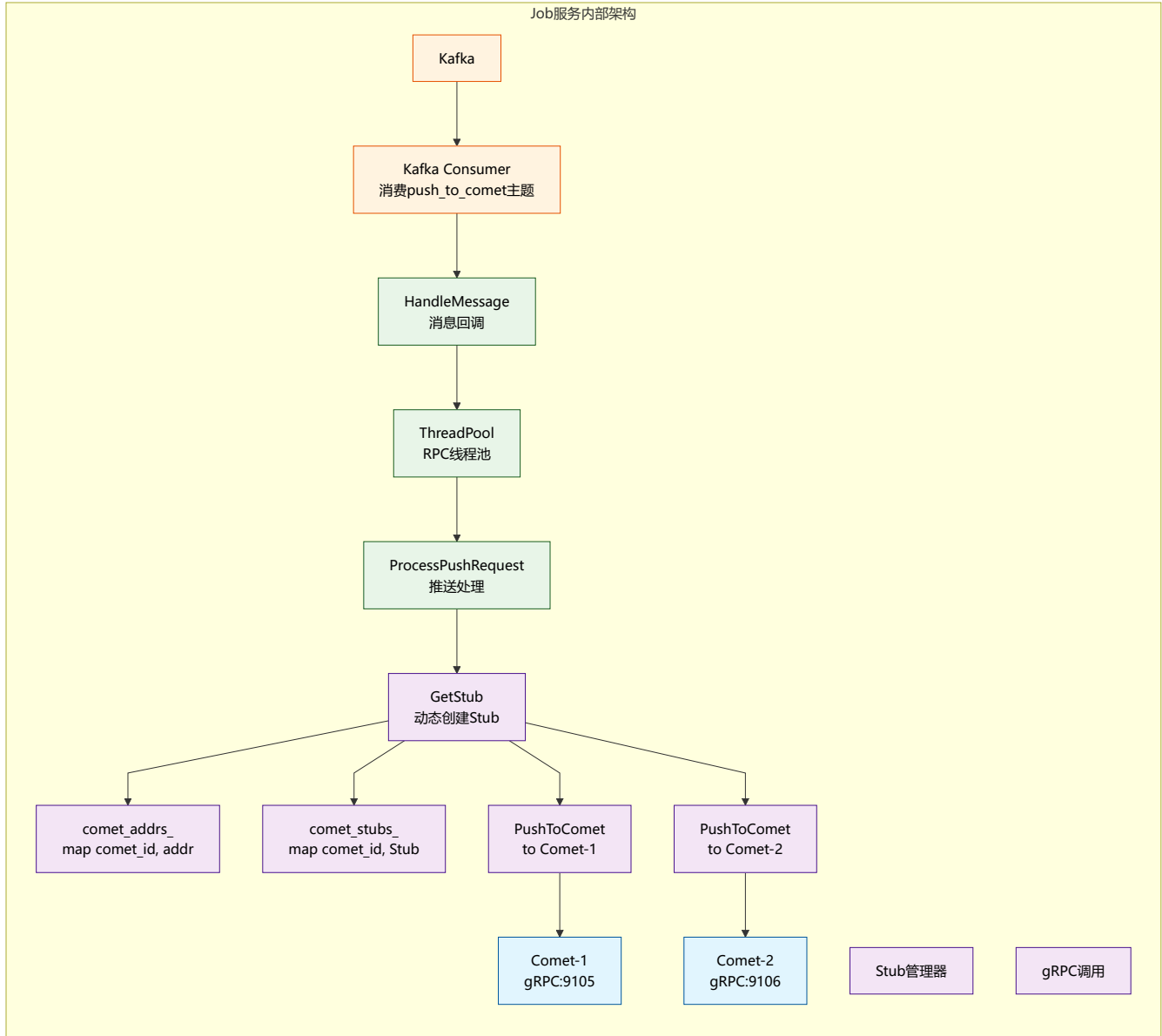
4.3. Job服务架构

4.3.1 核心组件

组件	类型	职责
KafkaConsumer	消息队列	从 push_to_comet 主题消费推送任务
ThreadPool	线程池	异步处理RPC调用（避免阻塞消费线程）
Stub管理器	gRPC客户端	动态管理多个Comet节点的gRPC连接
comet_addr_	配置映射	map<comet_id, grpc_addr>

组件	类型	职责
comet_stubs_	连接缓存	map<comet_id, Stub>

4.3.2 job架构图



4.3.3 关键实现

服务启动

```
// job/main.cpp

int main(int argc, char* argv[]) {
    // 1. 加载配置
    Config cfg;
    if (!LoadConfig(argv[1], &cfg)) {
```

```

        return 1;
    }

    // 2. 创建JobRunner
    JobRunner runner(cfg);
    if (!runner.Init()) {
        LOG_ERROR << "Failed to init JobRunner";
        return 1;
    }

    // 3. 启动（Kafka消费循环）
    runner.Start(); // 阻塞，直到Stop()

    return 0;
}

```

JobRunner初始化

```

// job/service.cpp

bool JobRunner::Init() {
    // 1. 解析Comet目标列表
    ParseCometTargets();
    // 配置示例: comet_targets=comet-1=127.0.0.1:9105,comet-2=127.0.0.1:9106
    // 解析后: comet_addrs_["comet-1"] = "127.0.0.1:9105"

    // 2. 初始化Kafka Consumer
    if (!consumer_.Init(cfg_.kafka_brokers,
                        cfg_.kafka_push_topic,
                        cfg_.kafka_consumer_group)) {
        LOG_ERROR << "Failed to init kafka consumer";
        return false;
    }

    // 3. 设置消息回调
    consumer_.SetCallback(std::bind(&JobRunner::HandleMessage, this,
                                    std::placeholders::_1,
                                    std::placeholders::_2));

    // 4. 初始化RPC线程池
    rpc_pool_.Start(4); // 4个工作线程

    return true;
}

void JobRunner::ParseCometTargets() {
    // 配置格式: comet-1=127.0.0.1:9105,comet-2=127.0.0.1:9106
    std::string targets = cfg_.comet_targets;
    std::istringstream iss(targets);
    std::string item;

    while (std::getline(iss, item, ',')) {

```

```

        size_t pos = item.find('=');
        if (pos != std::string::npos) {
            std::string comet_id = item.substr(0, pos);
            std::string addr = item.substr(pos + 1);
            comet_addrs_[comet_id] = addr;
            LOG_INFO << "Comet target: " << comet_id << " -> " << addr;
        }
    }
}

```

动态Stub管理

```

CometService::Stub* JobRunner::GetStub(const std::string& comet_id) {
    std::lock_guard<std::mutex> lock(stub_mu_);

    // 1. 查找缓存的Stub
    auto it = comet_stubs_.find(comet_id);
    if (it != comet_stubs_.end()) {
        return it->second.get();
    }

    // 2. 根据comet_id获取地址
    auto addr_it = comet_addrs_.find(comet_id);
    if (addr_it == comet_addrs_.end()) {
        LOG_ERROR << "Unknown comet_id: " << comet_id;
        return nullptr;
    }

    // 3. 创建新的Channel和Stub
    auto channel = grpc::CreateChannel(addr_it->second,
                                       grpc::InsecureChannelCredentials());
    auto stub = CometService::NewStub(channel);
    auto* ptr = stub.get();
    comet_stubs_[comet_id] = std::move(stub);

    LOG_INFO << "Created gRPC stub for " << comet_id << " -> " << addr_it->second;
    return ptr;
}

```

消费与处理

```

void JobRunner::HandleMessage(const std::string& key, const std::string& value) {
    (void)key; // 当前未使用 kafka 消息的 key

    // 提交到线程池异步处理（避免阻塞Kafka消费线程）
    rpc_pool_.Submit([this, payload = value]() {
        PushToCometRequest req;
        if (!req.ParseFromString(payload)) {
            LOG_ERROR << "Failed to parse PushToCometRequest from kafka";
            return;
        }
        ProcessPushRequest(req);
    });
}

```

```

});
}

void JobRunner::ProcessPushRequest(const PushToCometRequest& req) {
    // 1. 确定目标 Comet 节点列表
    std::vector<std::string> comet_ids;
    if (req.comet_ids_size() > 0) {
        // 使用请求中指定的 Comet 节点
        comet_ids.reserve(req.comet_ids_size());
        for (const auto& cid : req.comet_ids()) {
            comet_ids.push_back(cid);
        }
    } else {
        // 未指定时，广播到所有已配置的 Comet 节点
        for (const auto& kv : comet_addrs_) {
            comet_ids.push_back(kv.first);
        }
    }

    // 2. 检查是否有可用的 Comet 节点
    if (comet_ids.empty()) {
        LOG_ERROR << "No comet targets available, drop msg_id="
            << req.message().msg_id();
        return;
    }

    // 3. 遍历目标 Comet 节点，依次发送推送请求
    for (const auto& cid : comet_ids) {
        // 3.1 获取该 Comet 节点的 gRPC Stub
        CometService::Stub* stub = GetStub(cid);
        if (!stub) {
            LOG_ERROR << "Unknown comet_id " << cid
                << " for msg_id=" << req.message().msg_id();
            continue;
        }

        // 3.2 构造发送给单个 Comet 的请求（关键：按 comet_id 过滤 targets，避免重复投递）
        PushToCometRequest req2;
        *req2.mutable_message() = req.message();
        req2.add_comet_ids(cid);
        for (const auto& t : req.targets()) {
            if (t.comet_id().empty() || t.comet_id() == cid) {
                *req2.add_targets() = t;
            }
        }

        // 若过滤后没有目标用户，直接跳过，避免无意义 RPC
        if (req.targets_size() > 0 && req2.targets_size() == 0) {
            LOG_INFO << "Skip PushToComet to comet_id=" << cid
                << " because no matching targets. msg_id="
                << req.message().msg_id();
            continue;
        }
    }
}

```

```

// 3.3 发起 gRPC 调用
PushToCometReply reply;
grpc::ClientContext ctx;
LOG_INFO << "PushToComet to comet_id=" << cid
          << ", msg_id=" << req.message().msg_id();
auto status = stub->PushToComet(&ctx, req2, &reply);

// 3.4 检查 RPC 调用是否成功
if (!status.ok()) {
    LOG_ERROR << "PushToComet RPC failed to comet_id=" << cid << ": "
              << status.error_message();
    continue;
}

// 3.5 检查业务层返回的错误码
if (reply.error().code() != 0) {
    LOG_ERROR << "Comet response error to comet_id=" << cid << ": "
              << reply.error().message();
}
}
}

```

4.4 总结

三大服务对比

维度	Comet	Logic	Job
网络协议	WebSocket + gRPC	gRPC + HTTP	gRPC客户端
并发模型	muduo Reactor + IO线程池	muduo Reactor + gRPC独立线程	Kafka Pull + RPC线程池
核心存储	内存 (user_conns_)	Redis + MySQL	无状态
主要职责	长连接管理、推送	业务逻辑、路由决策	消息分发
扩展方式	水平扩展 (多实例)	水平扩展 (多实例)	水平扩展 (消费者组)

调用链路总结

上行: 客户端 → Comet (WebSocket) → Logic (gRPC) → Kafka
 下行: Kafka → Job (Consumer) → Comet (gRPC) → 客户端 (WebSocket)